

Visione Artificiale

Raffaella Lanza

Where are we?

First part: Image formation and Early vision

- Image formation
 - Geometric Camera Models
 - Color spaces
- Image Processing
 - Punctual and spatial processing
 - Feature Extraction
- Reconstruction
 - Camera calibration
 - Stereo Vision
 - Structure from Motion and RGB-d Cameras
 - Optical flow and Tracking

Second part: Machine learning for CV

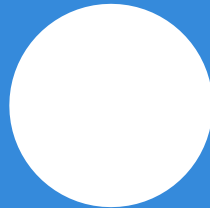
- Linear Neural Network
- Multi Layer Perceptron
- Convolutional Neural Networks
- **Recurrent Neural Networks**
- Transformers
- Variational Auto-Encoders
- Generative Adversarial Networks
- Graph Neural Networks
- Self-supervised learning
- Vision Language Models

Deep Sequence Modeling

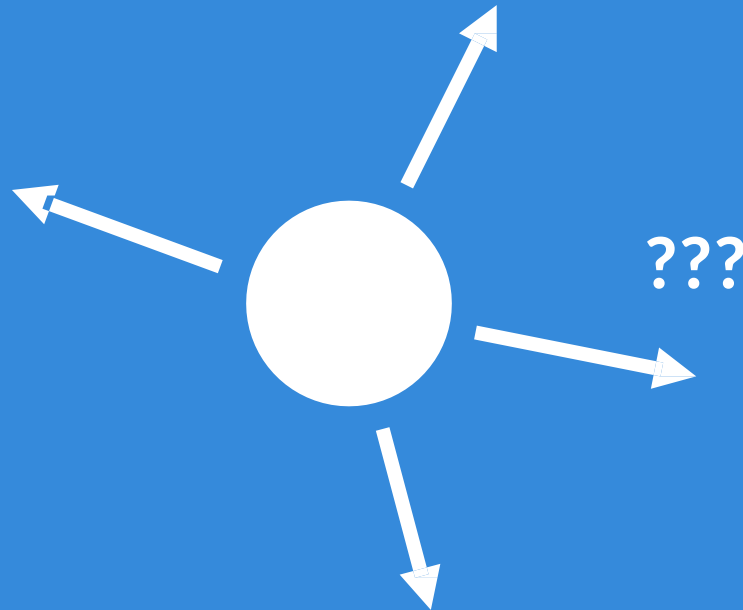
Raffaella Lanzaarotti

Chapter 9, d2l
attached notes

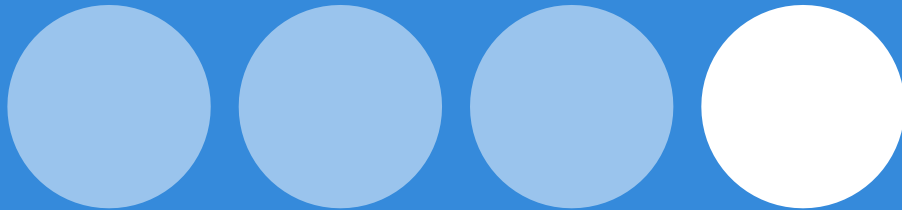
Given an image of a ball,
can you predict where it will go next?



Given an image of a ball,
can you predict where it will go next?



Given an image of a ball,
can you predict where it will go next?



Given an image of a ball,
can you predict where it will go next?



Sequences in the wild



Audio

Sequences in the wild



Audio

Sequences in the wild

character:

6.S191 Introduction to Deep Learning

word:

Text

Sequences in the wild

character:

6 . S | 9 |

word:

Introduction to Deep Learning

Sequences in the wild

Many others contexts:

- analysis of **medical signals** like ECGs
- financial market and stock trend prediction
- processing genomic data
- pattern in climate
- in Computer Vision: given the behavior of an object for a certain period, predict its behavior in next frames



Sequences in the wild

Many others contexts:

- analysis of medical signals like ECGs
- **financial market and stock trend prediction**
- processing genomic data
- pattern in climate
- in Computer Vision: given the behavior of an object for a certain period, predict its behavior in next frames



Sequences in the wild

Many others contexts:

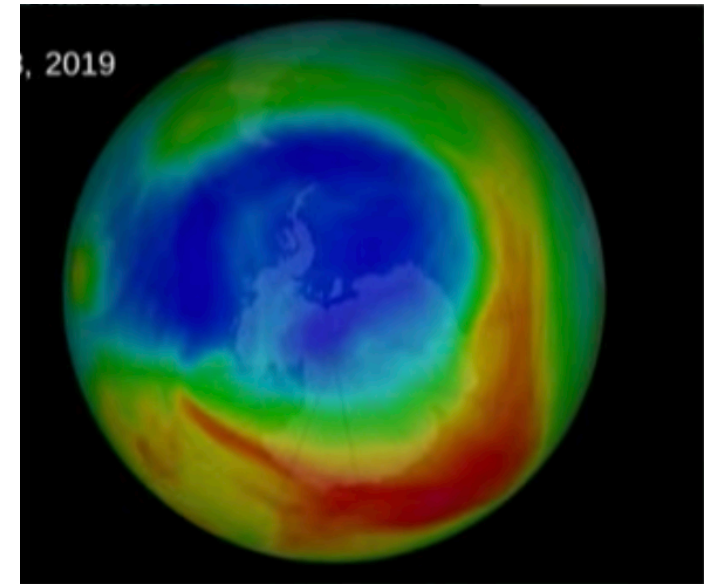
- analysis of medical signals like ECGs
- financial market and stock trend prediction
- **processing genomic data**
- pattern in climate
- in Computer Vision: given the behavior of an object for a certain period, predict its behavior in next frames



Sequences in the wild

Many others contexts:

- analysis of medical signals like ECGs
- financial market and stock trend prediction
- processing genomic data
- **pattern in climate**
- in Computer Vision: given the behavior of an object for a certain period, predict its behavior in next frames



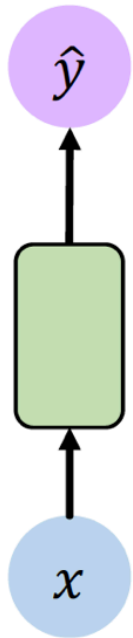
Sequences in the wild

Many others contexts:

- analysis of medical signals like ECGs
- financial market and stock trend prediction
- processing genomic data
- pattern in climate
- **in Computer Vision:** given the behavior of an object for a certain period, predict its behavior in next frames

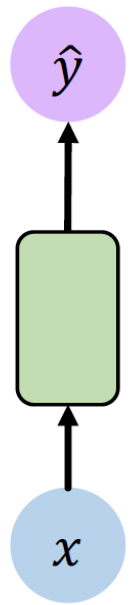


Standard feed-forward neural network

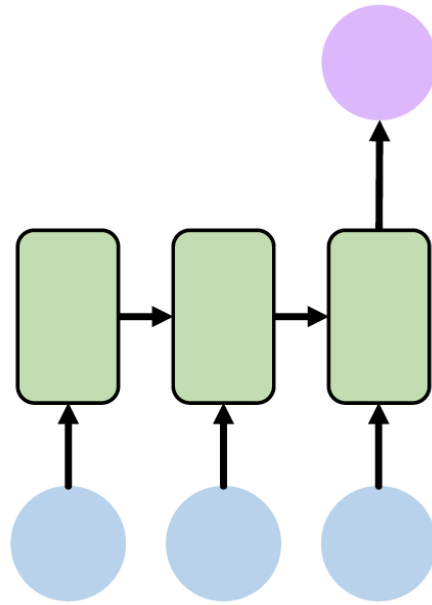


One to One
Instance Classification

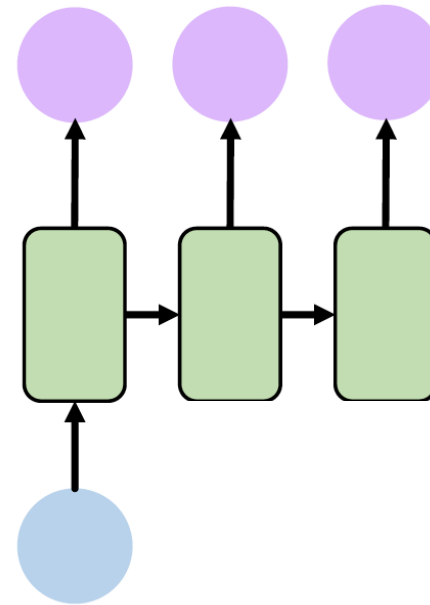
Recurrent neural networks: sequence modeling



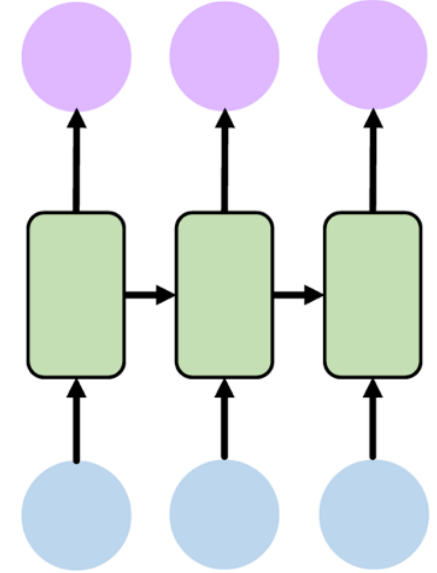
One to One
Instance Classification



Many to One
Sentiment Classification



One to Many
Image Captioning



Many to Many
Machine translation

Sequence modeling: design criteria

To model sequences, we need to:

1. Handle **variable-length** sequences
2. Track **long-term** dependencies
3. Maintain information about **order**
4. **Share parameters** across the sequence

A Sequence Modeling Problem: Predict the Next Word

A sequence modeling problem: predict the next word

“This morning I took my cat for a walk.”

A sequence modeling problem: predict the next word

“This morning I took my cat for a walk.”

given these words

predict the
next word

(

Representing Language to a Neural Network

Representing Language to a Neural Network

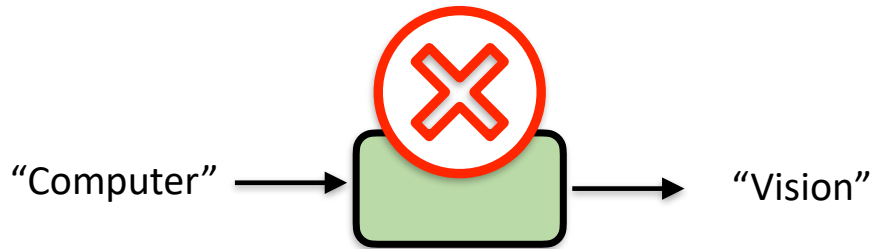
“This morning I took my cat for a walk.”

given these words

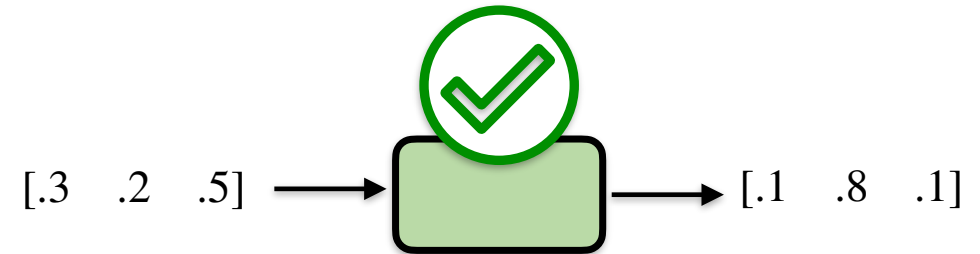
predict the
next word



Representing Language to a Neural Network



Neural Networks cannot interpret words



Neural Networks require numerical inputs

Embedding: transform indexes into a vector of fixed size

morning
this walk
cat a
on dog

1. Vocabulary
Corpus of words

a → 1
cat → 2
...
walk → N

2. Indexing
Word to index

One-hot embedding
"cat" = [0, 1, 0, 0, 0, 0]
↑
i-th index

3. Embedding:
Index to fixed-sized vector

Representing Language to a Neural Network



Handle variable-length sequences

The food was great

vs.

We visited a restaurant for lunch

vs.

We were very hungry but cleaned the house before eating

Treat long-term dependencies

"France is where I grew up, but now I live in Boston. I speak a fluent _____"

We need information from the distant past
to accurately predict the current word



Capture information about the order



The food was good, not bad at all.

vs.

The food was bad, not good at all.



Sequence modeling: design criteria

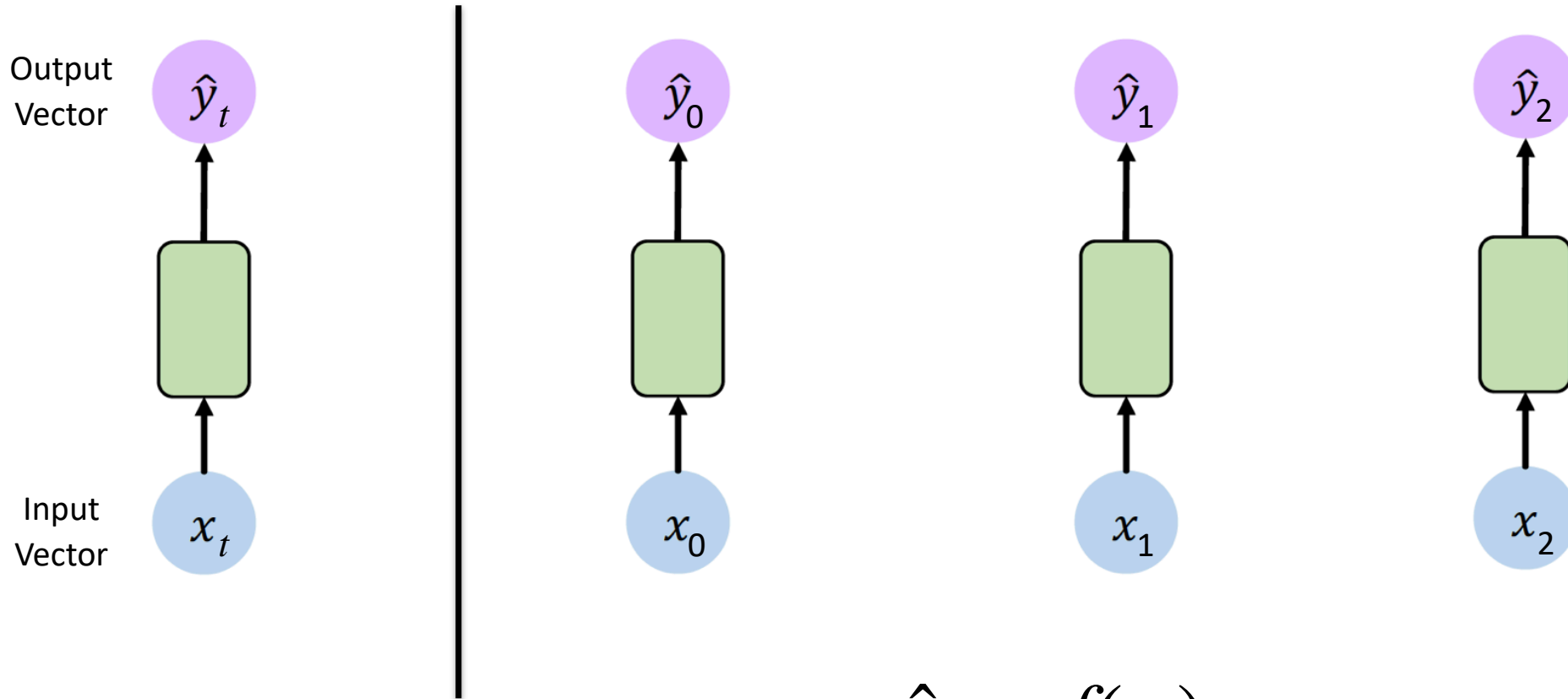
To model sequences, we need to:

1. Handle **variable-length** sequences
2. Track **long-term** dependencies
3. Maintain information about **order**
4. **Share parameters** across the sequence

Recurrent Neural Networks (RNNs) meet
(partially) these sequence modeling design criteria

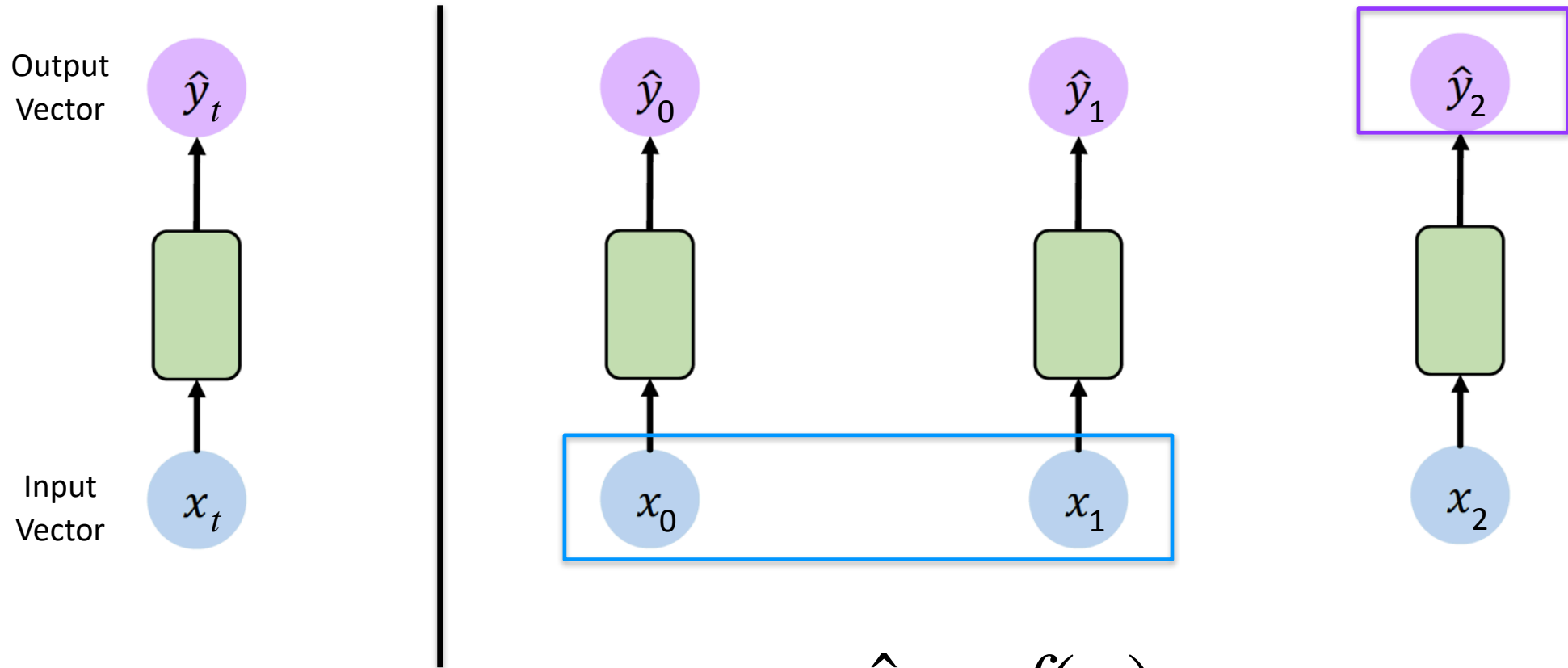
Recurrent Neural Networks (RNNs)

Handling Individual Time Steps



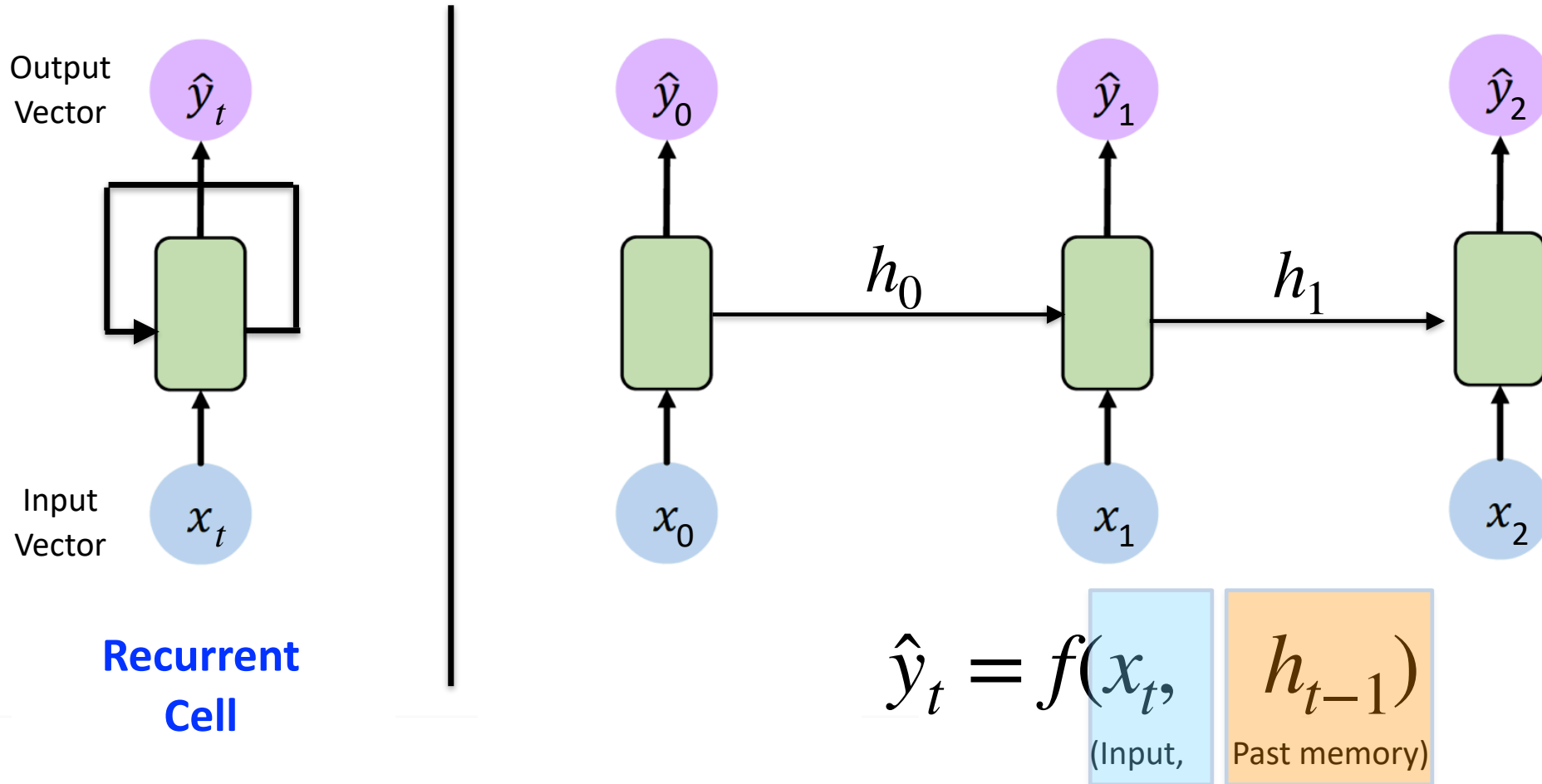
$$\hat{y}_t = f(x_t)$$

Handling Individual Time Steps

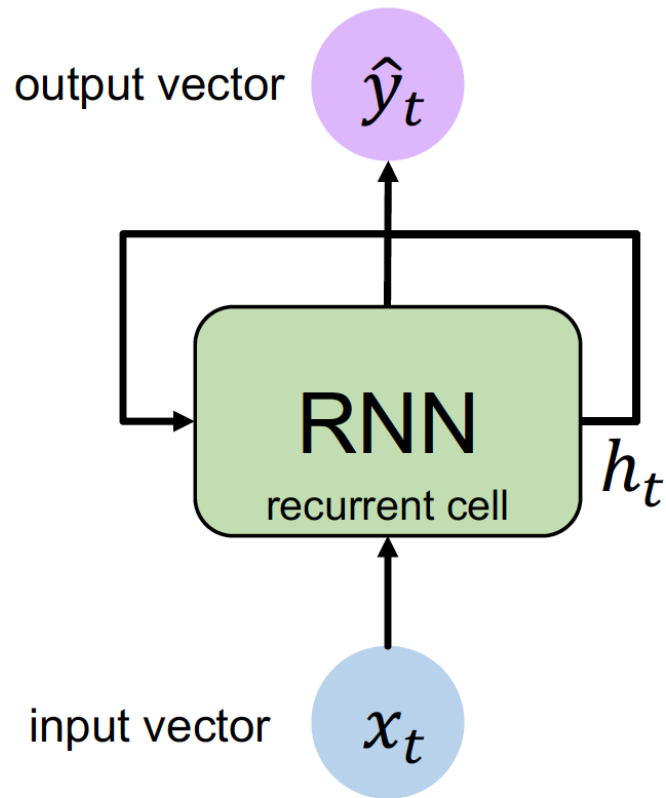


$$\hat{y}_t = f(x_t)$$

Neurons with Recurrence



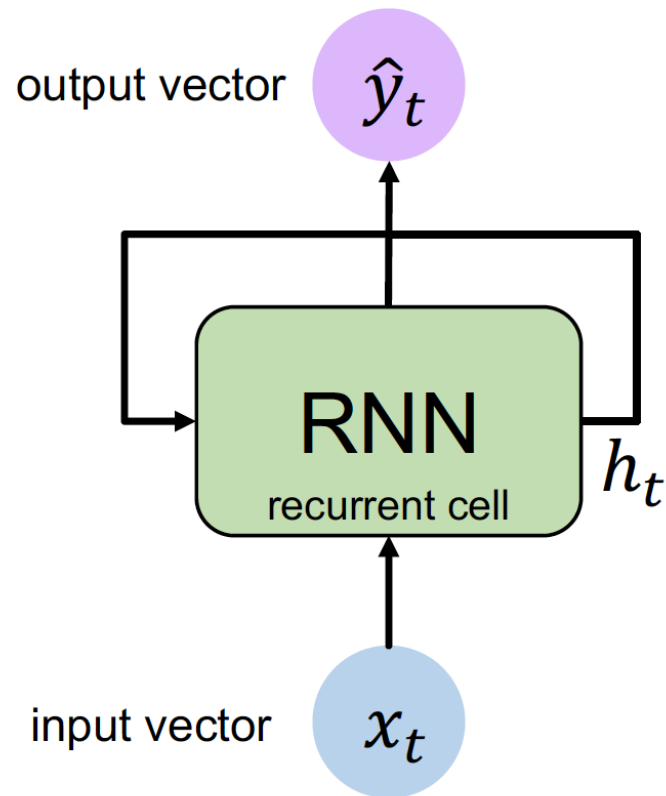
A recurrent neural network (RNN)



Recurrent:

information is passed internally from one time step to the next

A recurrent neural network (RNN)

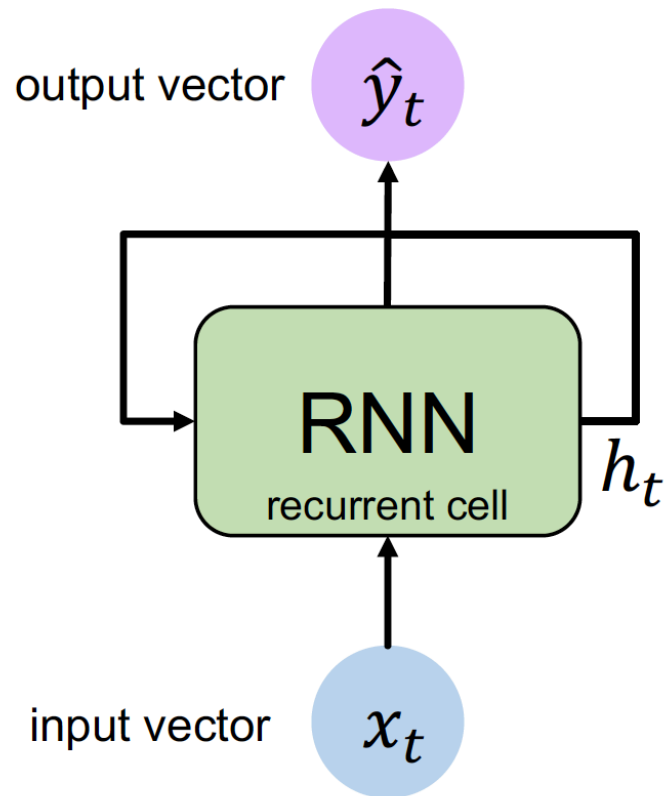


Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

cell state function old state input vector at
parameterized by W time step t

A recurrent neural network (RNN)



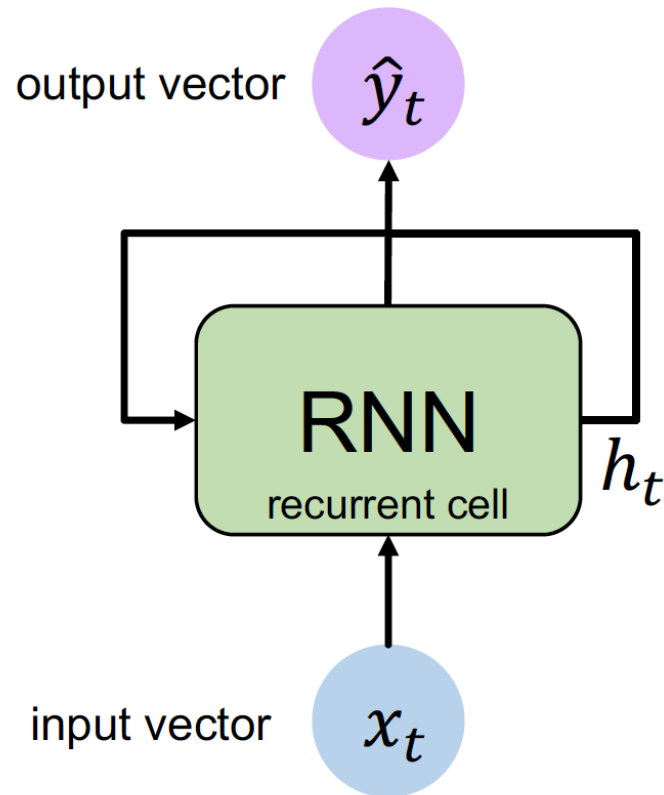
Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state function parameterized by W old state input vector at time step t

Note: the same function and set of parameters are used at every time step

RNN state update and output



Output Vector

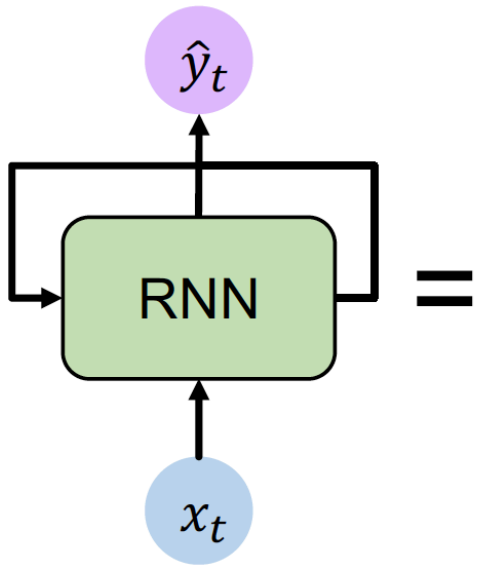
$$\hat{y}_t = \mathbf{W}_{hy}h_t$$

Hidden State

$$h_t = \tanh(\mathbf{W}_{hh}h_{t-1} + \mathbf{W}_{xh}x_t)$$

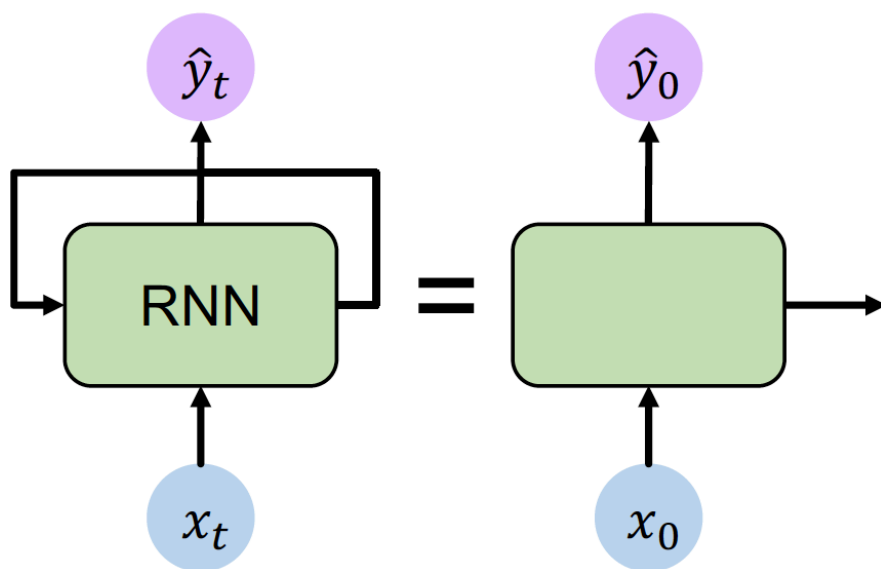
Input Vector

RNNs: computational graph across time



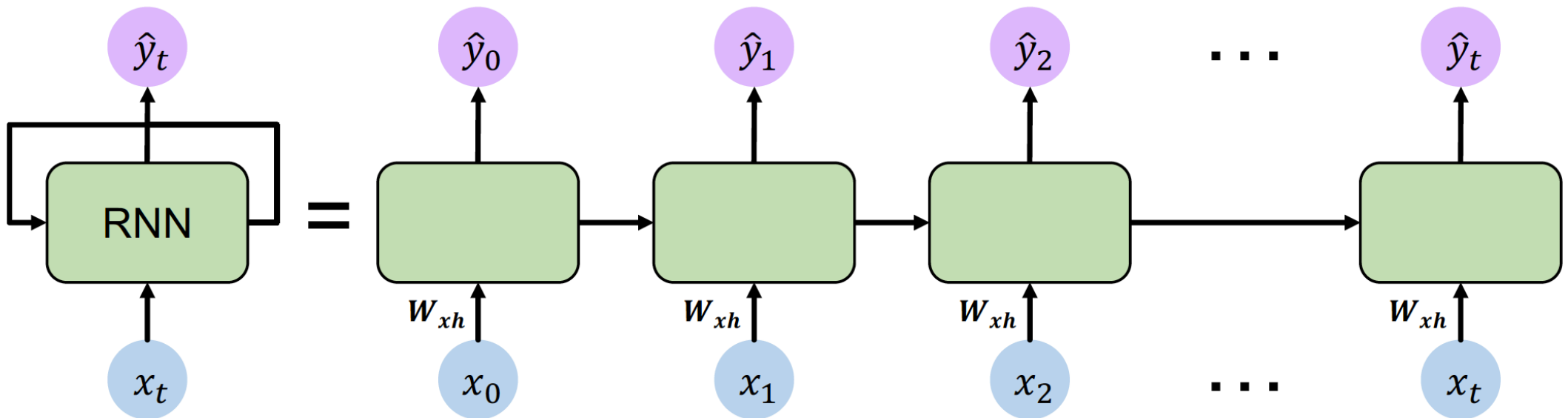
Represent as computational graph unrolled across time

RNNs: computational graph across time



RNNs: computational graph across time

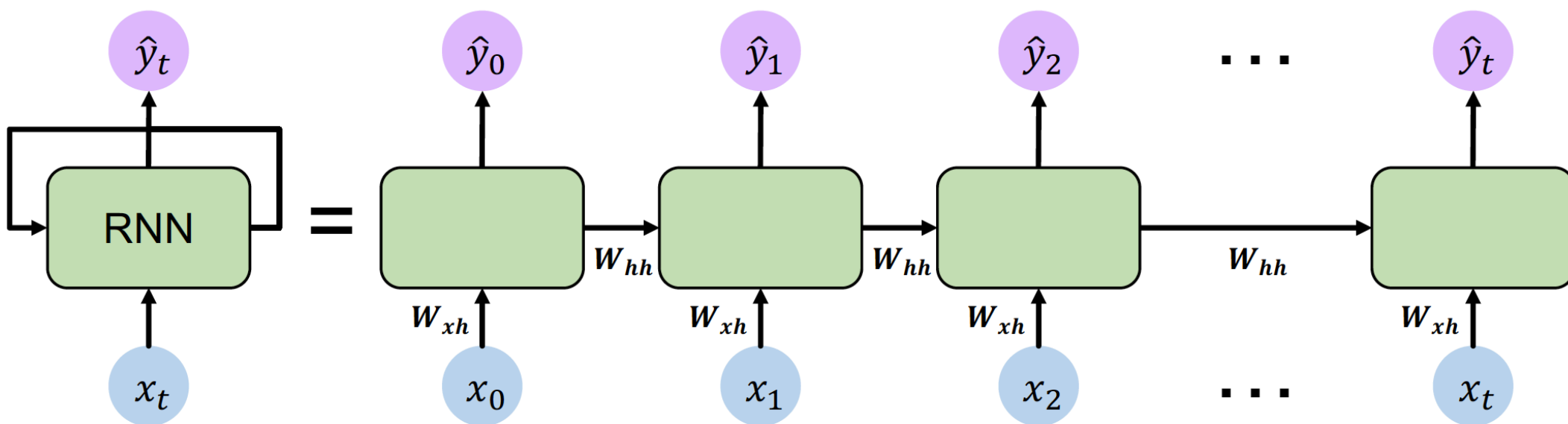
$$W_{xh}: x \rightarrow h$$



RNNs: computational graph across time

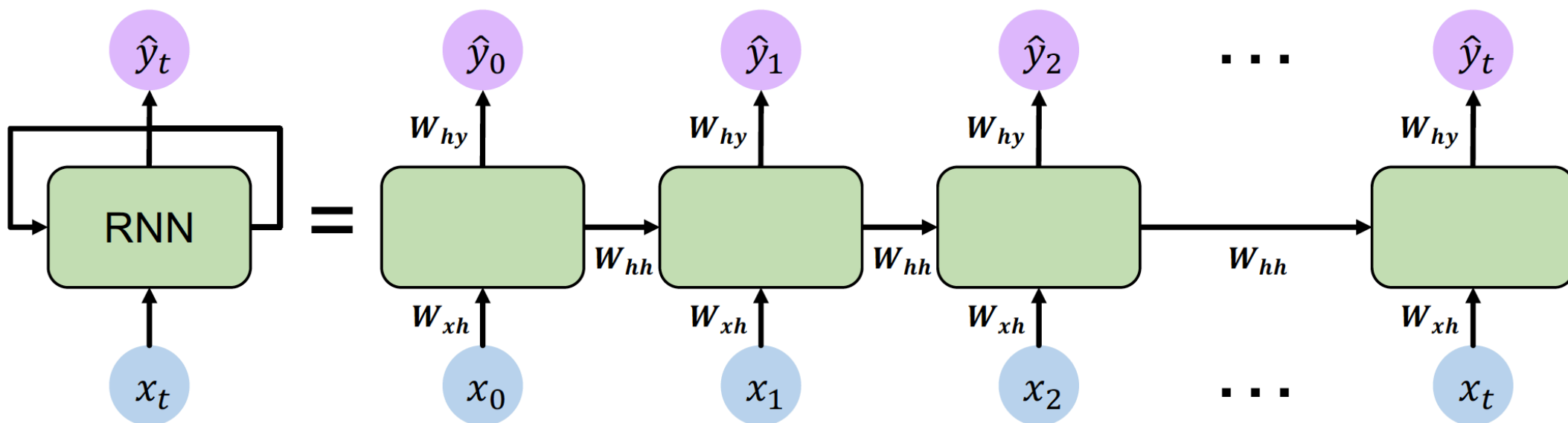
$$W_{xh}: x \rightarrow h$$

$$W_{hh}: h \rightarrow h$$



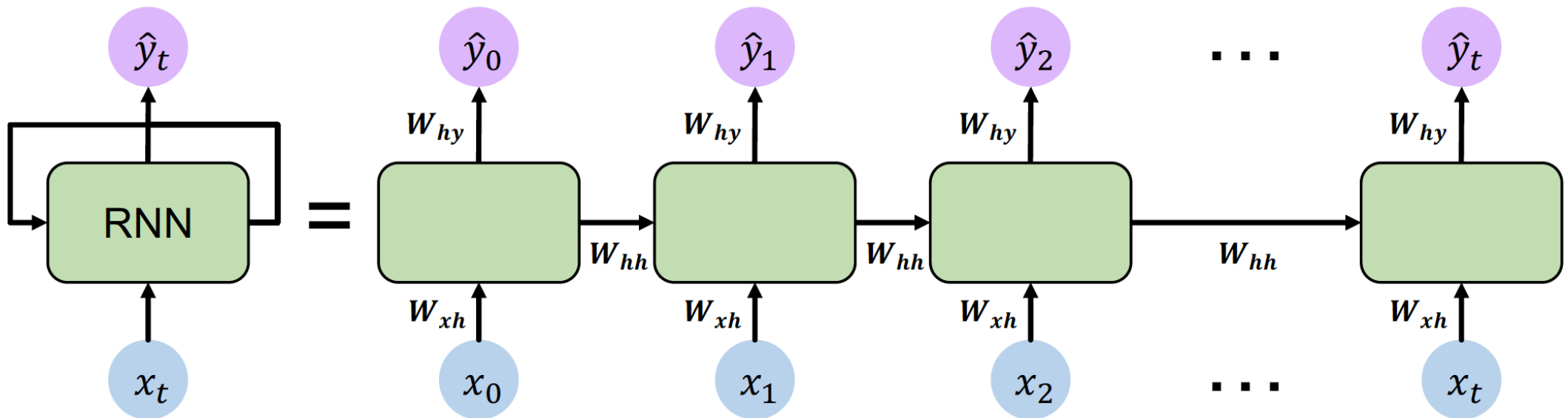
RNNs: computational graph across time

$$\begin{aligned}W_{xh}: x &\rightarrow h \\W_{hh}: h &\rightarrow h \\W_{hy}: h &\rightarrow y\end{aligned}$$



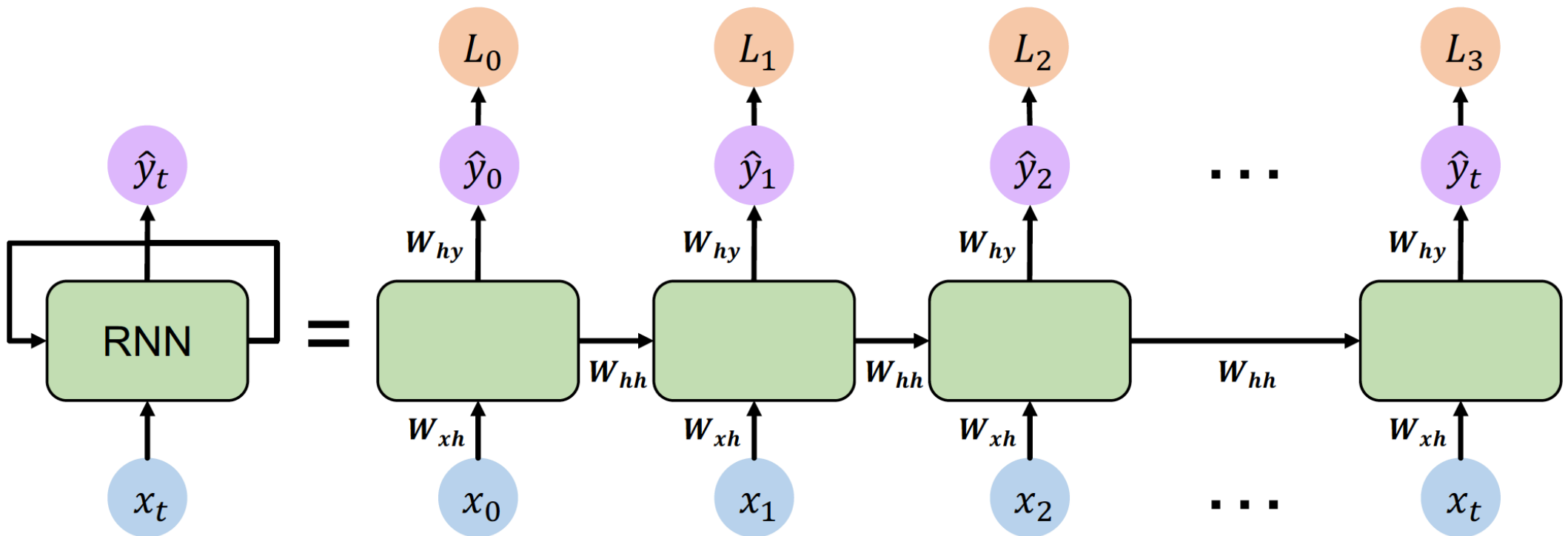
RNNs: computational graph across time

Re-use the **same weight matrices** at every time step

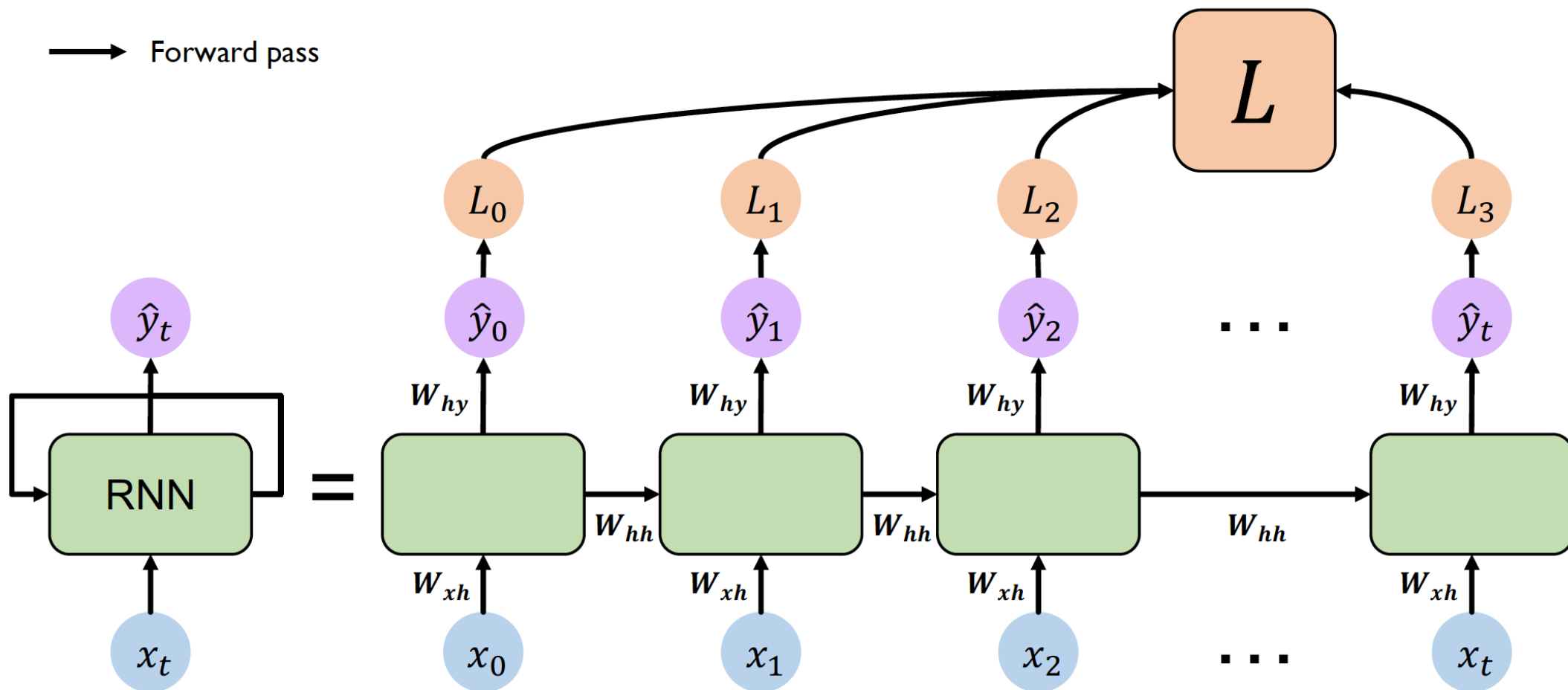


RNNs: computational graph across time

→ Forward pass

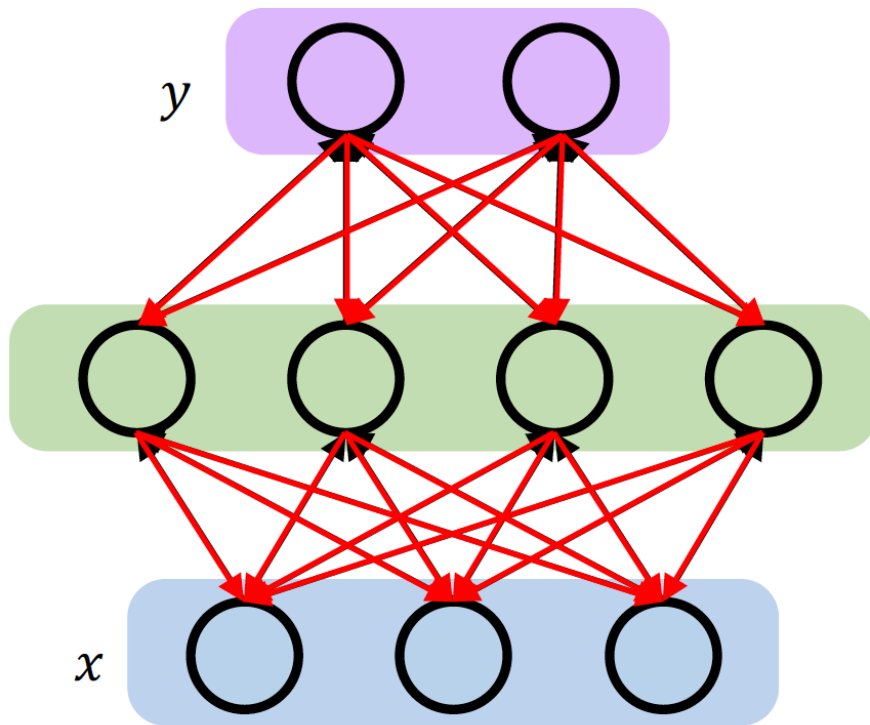


RNNs: computational graph across time



Backpropagation Through Time(BPTT)

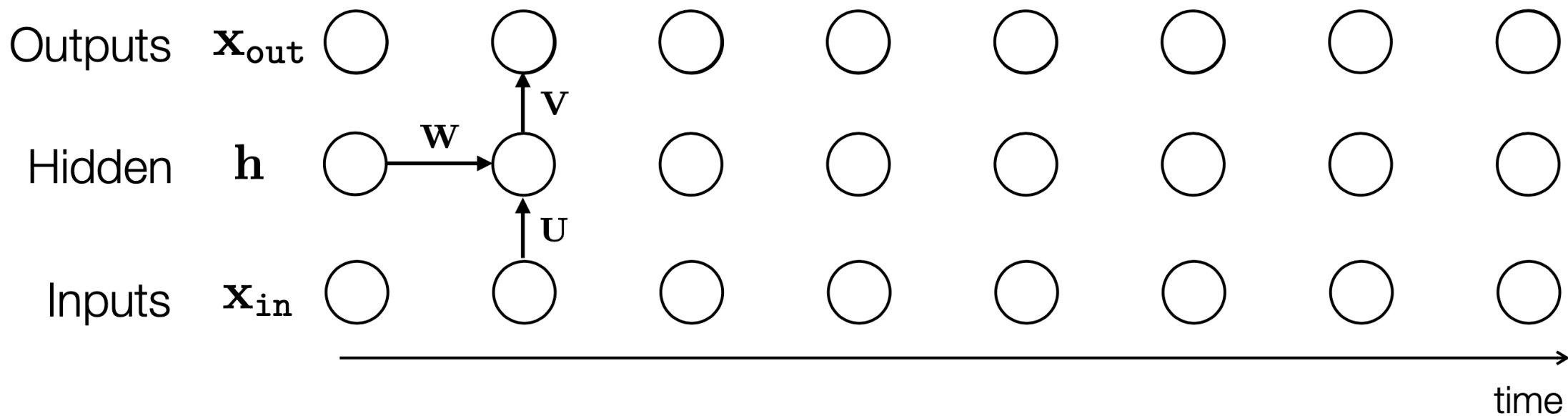
Recall: backpropagation in feed forward models



Backpropagation algorithm:

1. Take the derivative (gradient) of the loss with respect to each parameter
2. Shift parameters in order to minimize loss

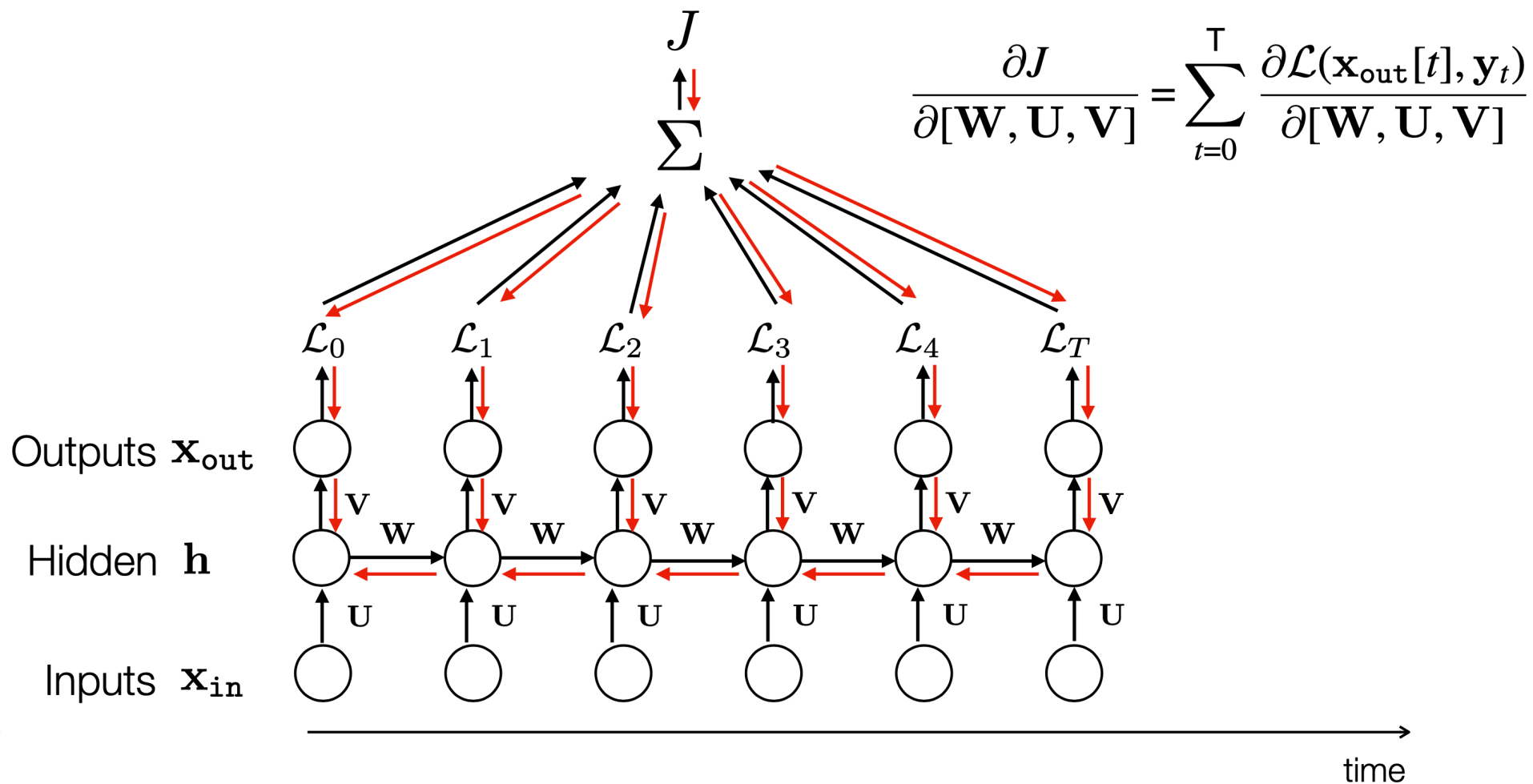
RNNs



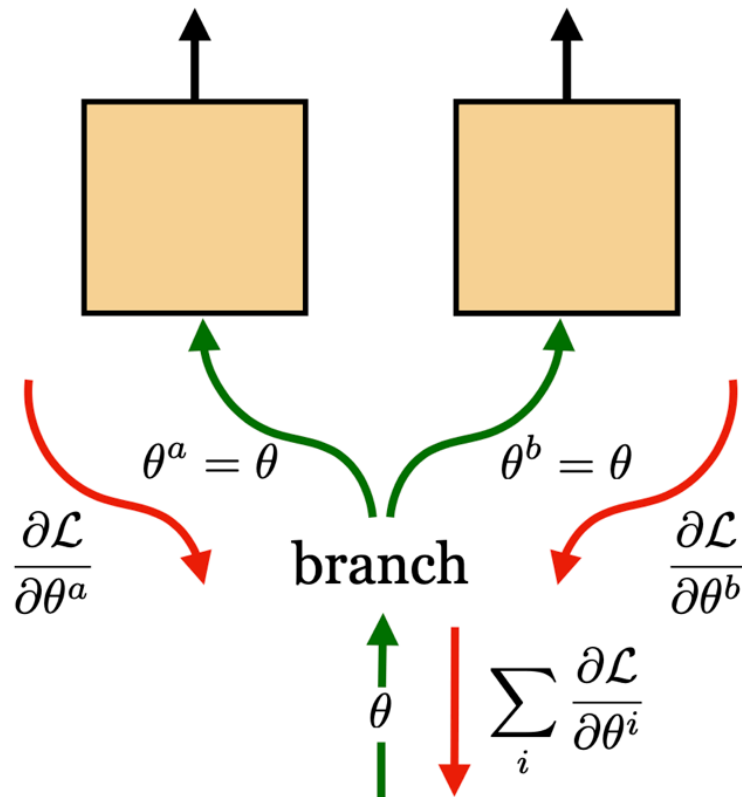
$$\mathbf{h}_t = \sigma_1(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_{\text{in}}[t] + \mathbf{b})$$

$$\mathbf{x}_{\text{out}}[t] = \sigma_2(\mathbf{V}\mathbf{h}_t + \mathbf{c})$$

Backpropagation through time



Parameter sharing



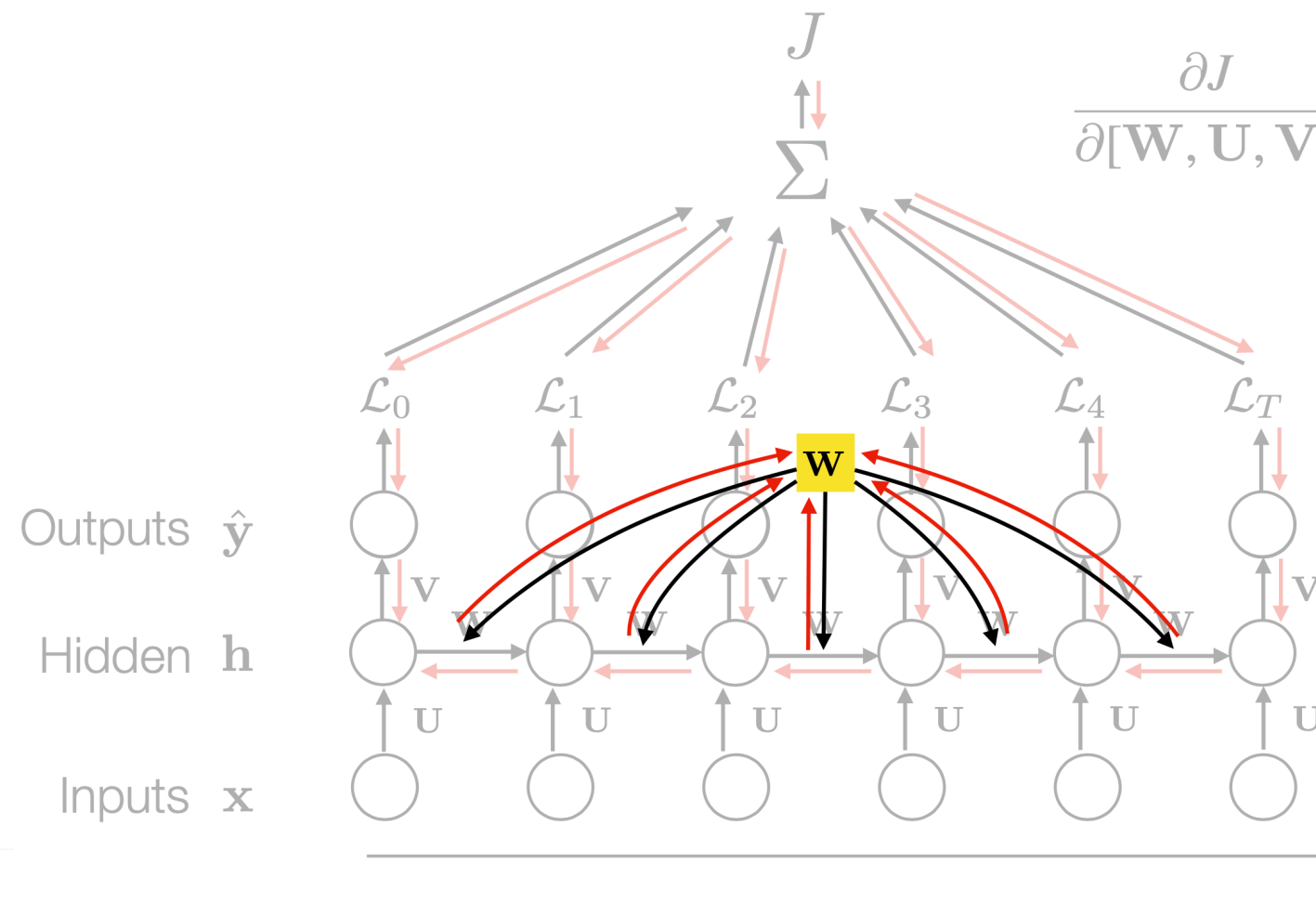
Suppose θ is used in more module of the network, here referred as θ^a, θ^b to distinguish them (CNN: kernel applied to more image localities, RNN same parameters at different time steps)

During training, the update of θ must take into account the gradients from all the places where the parameter was used.

The total gradient for θ is the sum of the gradients calculated for each position i where θ has had an impact on the loss function $\mathcal{L}$.

Backpropagation through time

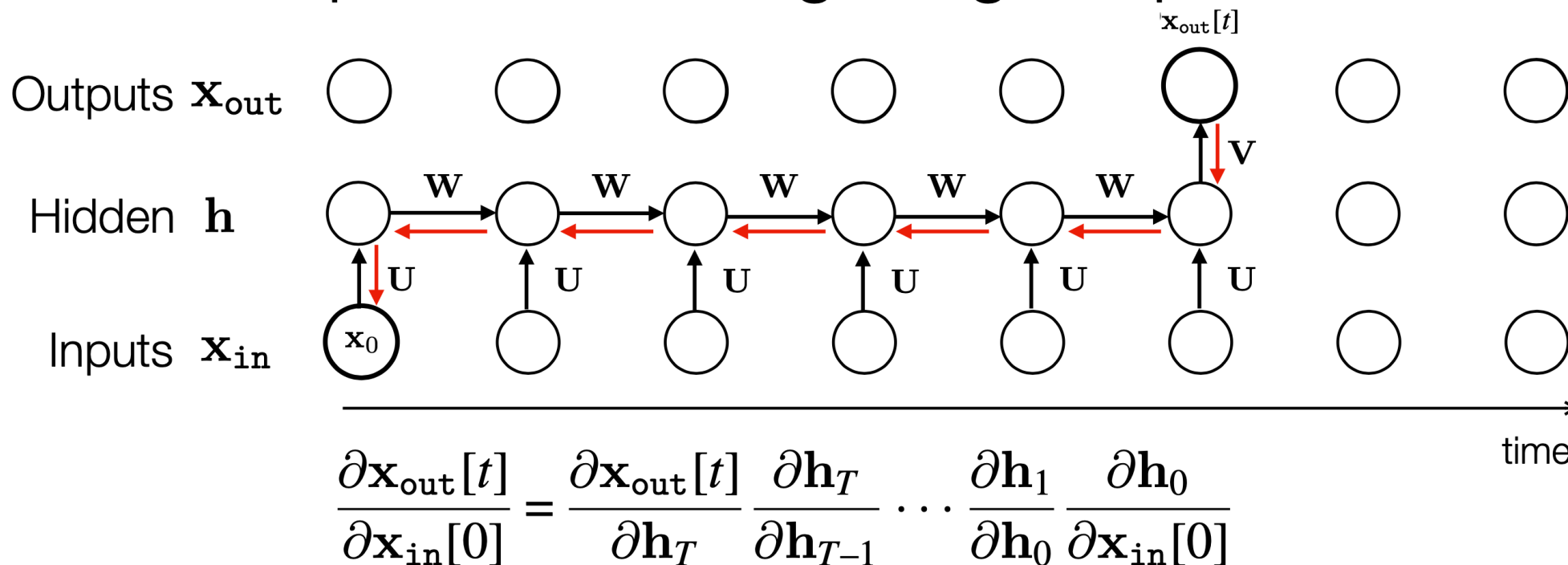
$$\frac{\partial J}{\partial [\mathbf{W}, \mathbf{U}, \mathbf{V}]} = \sum_{t=0}^T \frac{\partial \mathcal{L}(\mathbf{x}_{\text{out}}[t], \mathbf{y}_t)}{\partial [\mathbf{W}, \mathbf{U}, \mathbf{V}]}$$



$$\frac{\partial \mathcal{L}_t}{\partial \mathbf{W}} = \sum_i \frac{\partial \mathcal{L}_t}{\partial \mathbf{W}^i}$$

The calculation of the gradient for the weights $\mathbf{W}$ at time step t corresponds to the **sum of the gradients for each i** , that is, for **every instance in which the weight $\mathbf{W}$ has had an impact on the loss.**

The problem of long-range dependences



- Capturing long-range dependences requires propagating information through a long chain of dependences.
- Old observations are forgotten
- Stochastic gradients become high variance (noisy), and gradients may **vanish** or **explode**

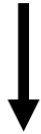
The problem of long-term dependencies

Why are vanishing gradients a problem?

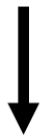
The problem of long-term dependencies

Why are vanishing gradients a problem?

Multiply many **small numbers** together



Errors due to further back time steps
have smaller and smaller gradients

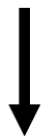


Bias network to capture short-term
dependencies

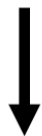
The problem of long-term dependencies

Why are vanishing gradients a problem?

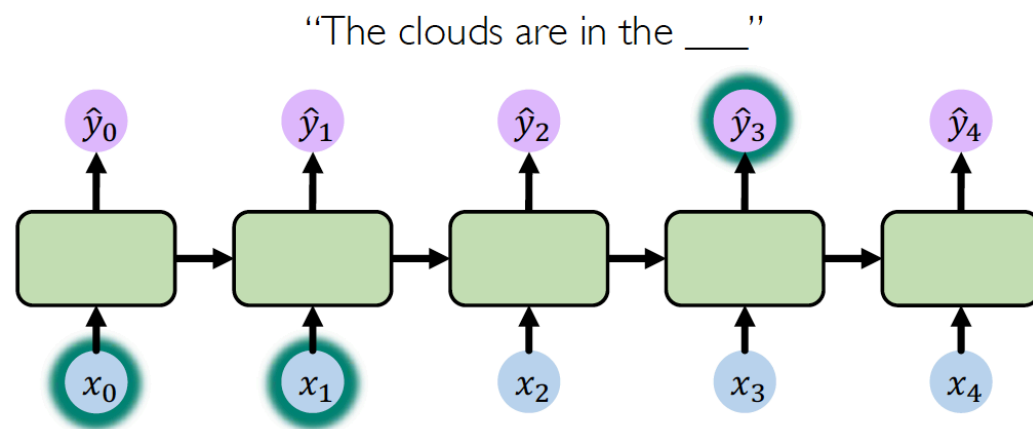
Multiply many **small numbers** together



Errors due to further back time steps
have smaller and smaller gradients



Bias parameters to capture short-term
dependencies



The problem of long-term dependencies

Why are vanishing gradients a problem?

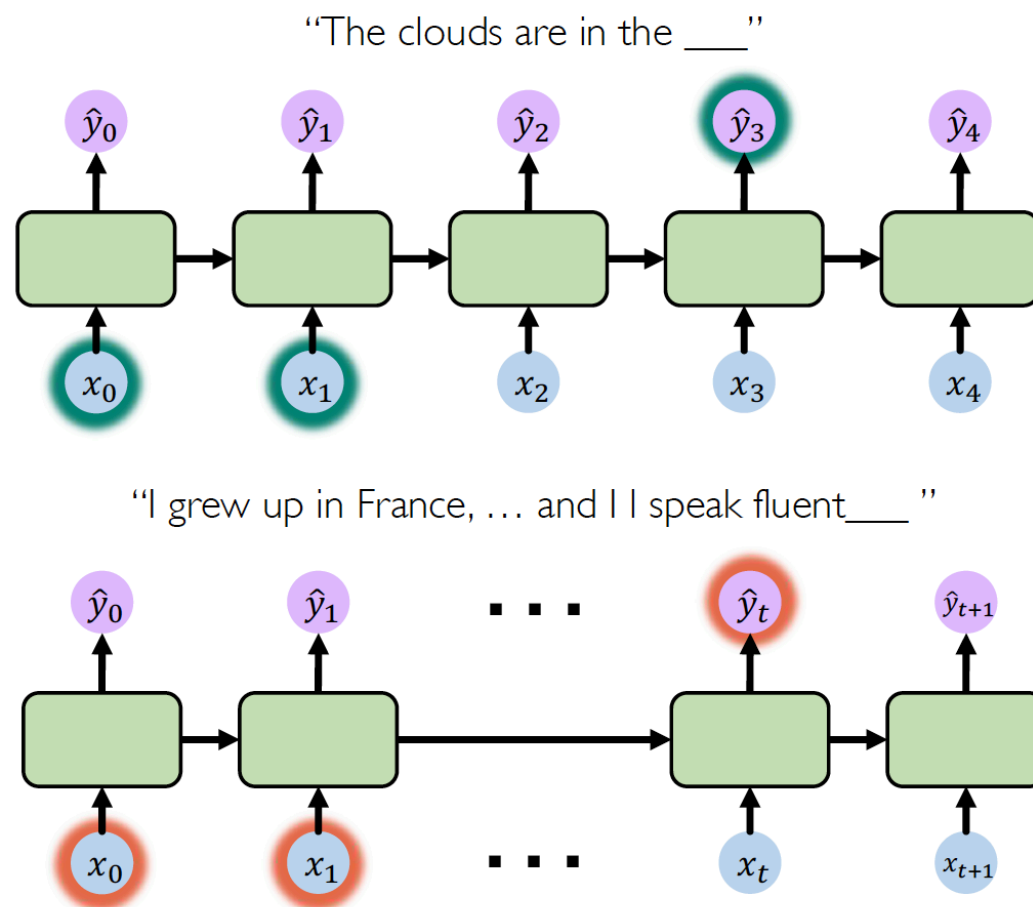
Multiply many **small numbers** together



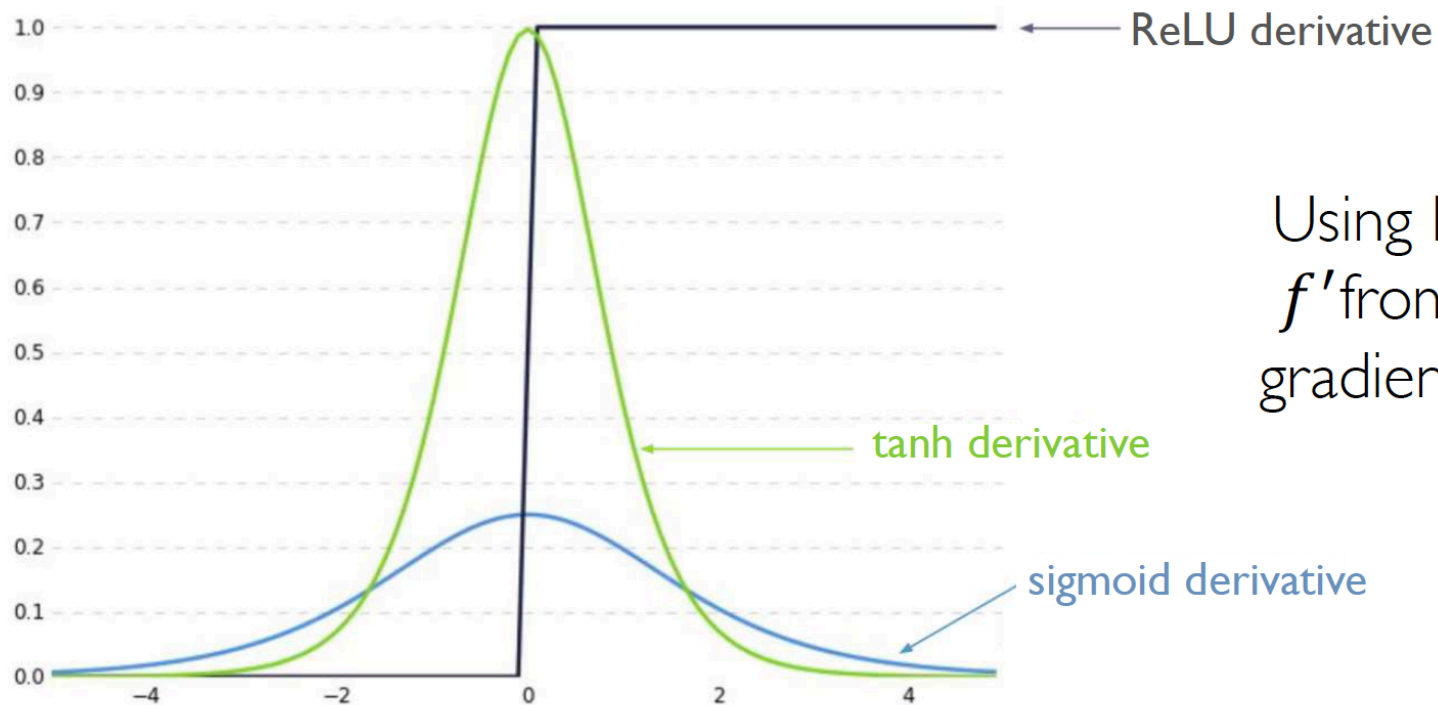
Errors due to further back time steps
have smaller and smaller gradients



Bias parameters to capture short-term
dependencies



Trick #1: activation functions



Using ReLU prevents f' from shrinking the gradients when $x > 0$

Trick #2: parameter initialization

Initialize **weights** to identity matrix

Initialize **biases** to zero

$$I_n = \begin{pmatrix} 1 & 0 & 0 & \cdots & 0 \\ 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \end{pmatrix}$$

This helps prevent the weights from shrinking to zero.

Trick #3: Gated Cells

Idea: use **gates** to selectively **add** or **remove** information within each recurrent unit with



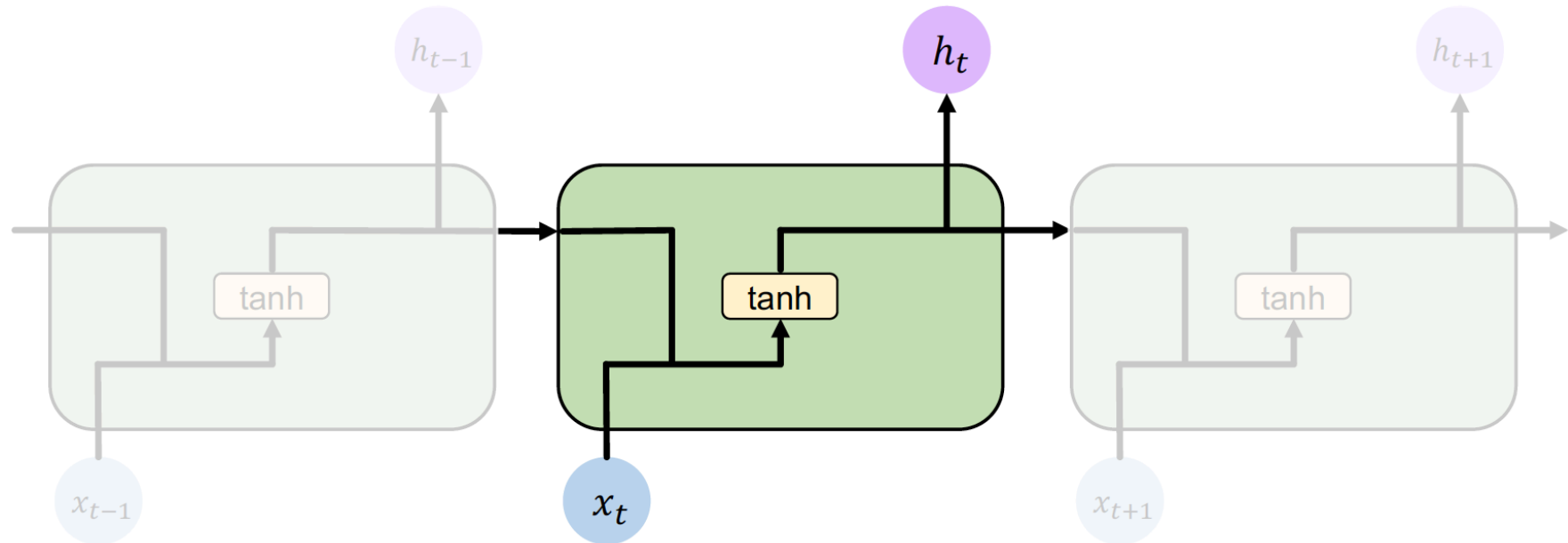
A special kind of RNN designed **to avoid forgetting**.

This way the default behavior is not to forget an old state.
Instead of forgetting by default, **the network has to learn what to forget**

Long Short Term Memory (LSTM) Networks

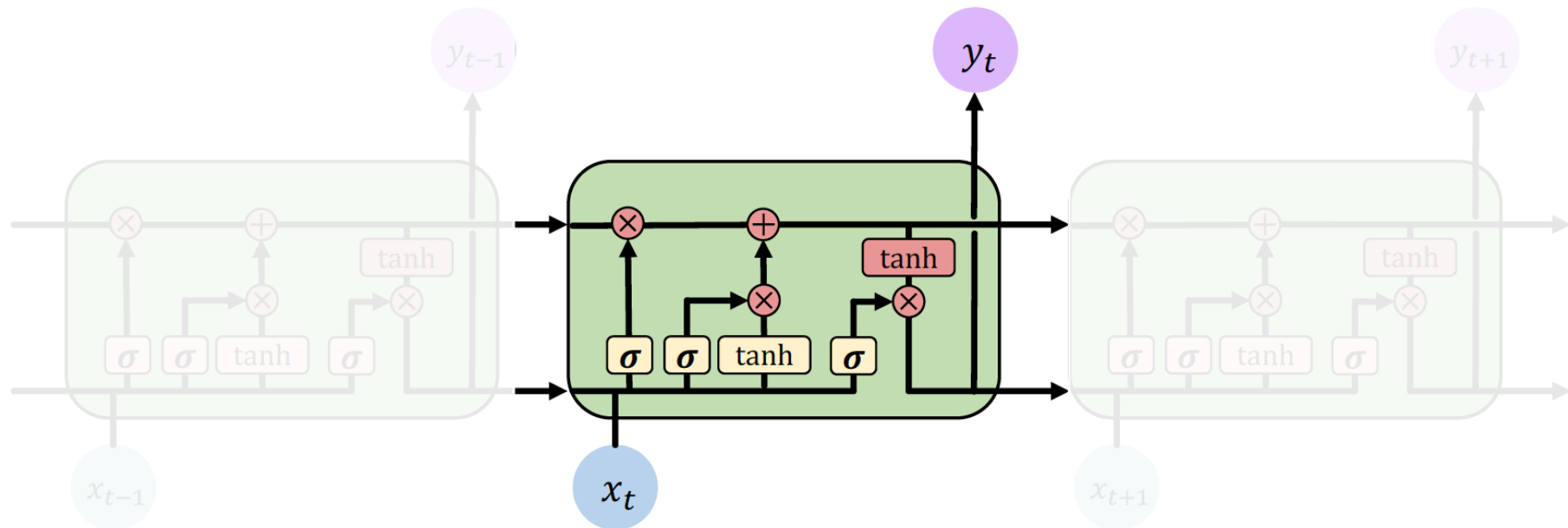
Standard RNN

In a standard RNN, repeating modules contain a **simple computation node**



Long Short Term Memory (LSTMs)

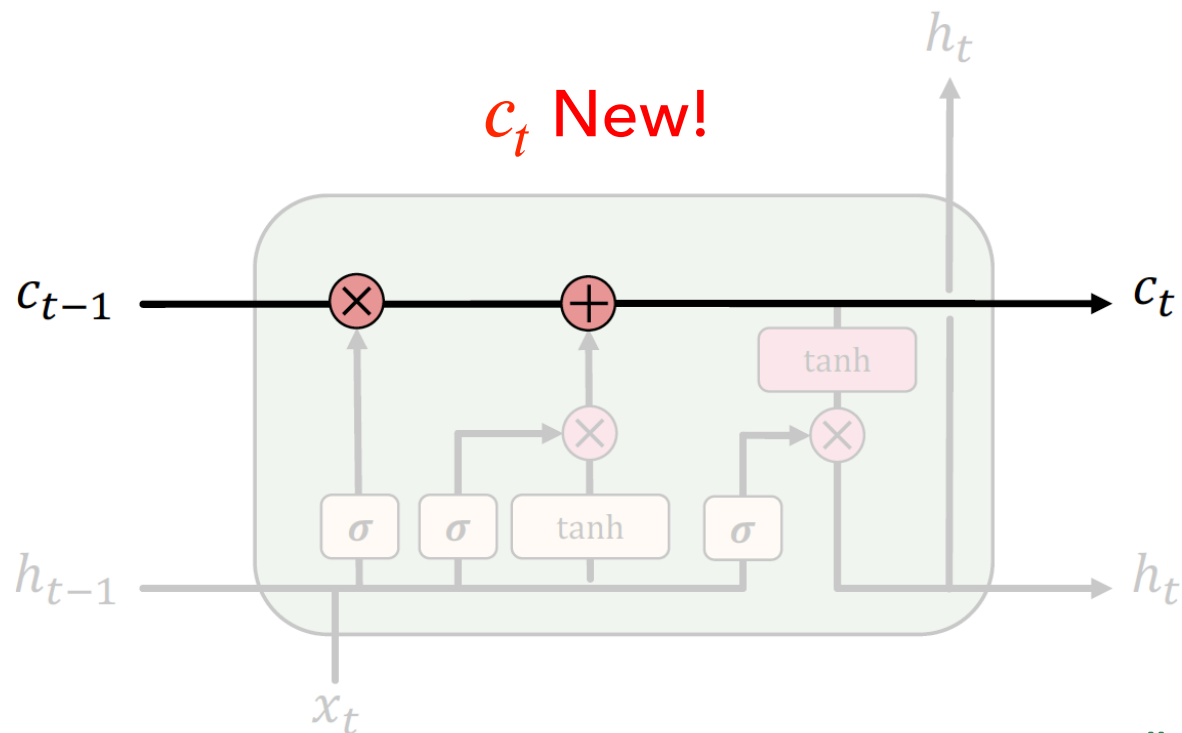
LSTM modules contain **computational blocks** that **control information flow**



LSTM cells are able to track information throughout many timesteps

Long Short Term Memory (LSTMs)

LSTMs maintain a **cell state** c_t where it's easy for information to flow

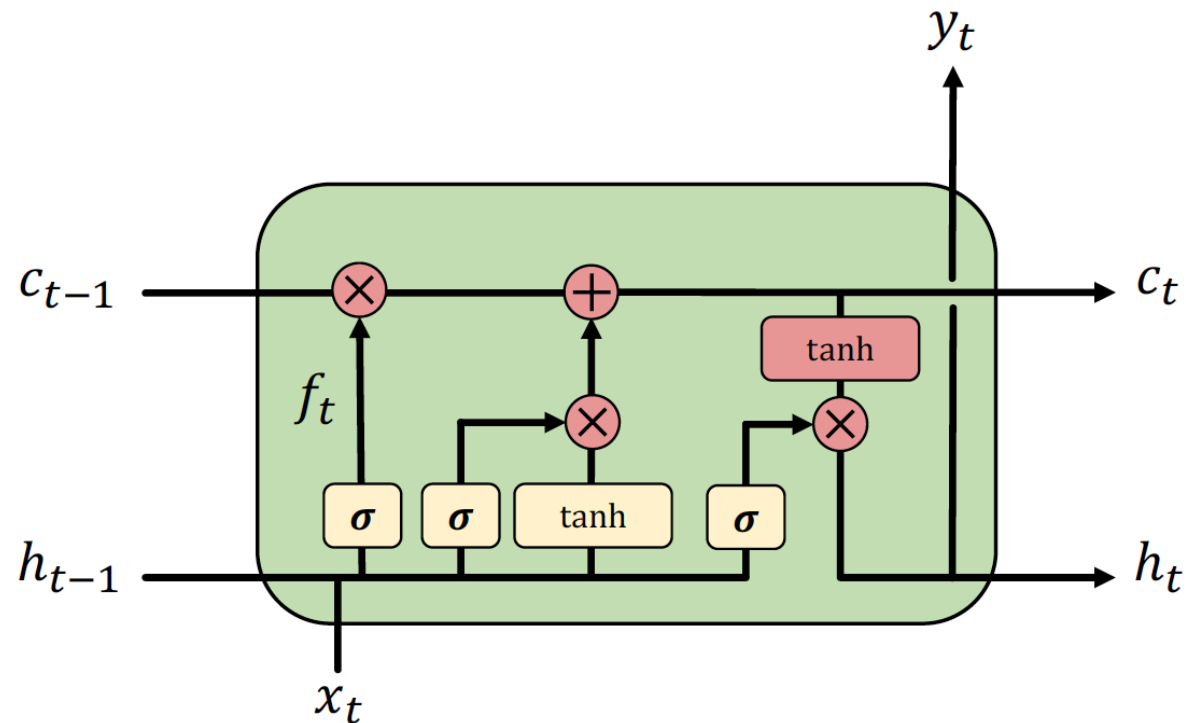


- **Cell State:** long term memory
- **Hidden State:** filtered output

Long Short Term Memory (LSTMs)

How do LSTMs work?

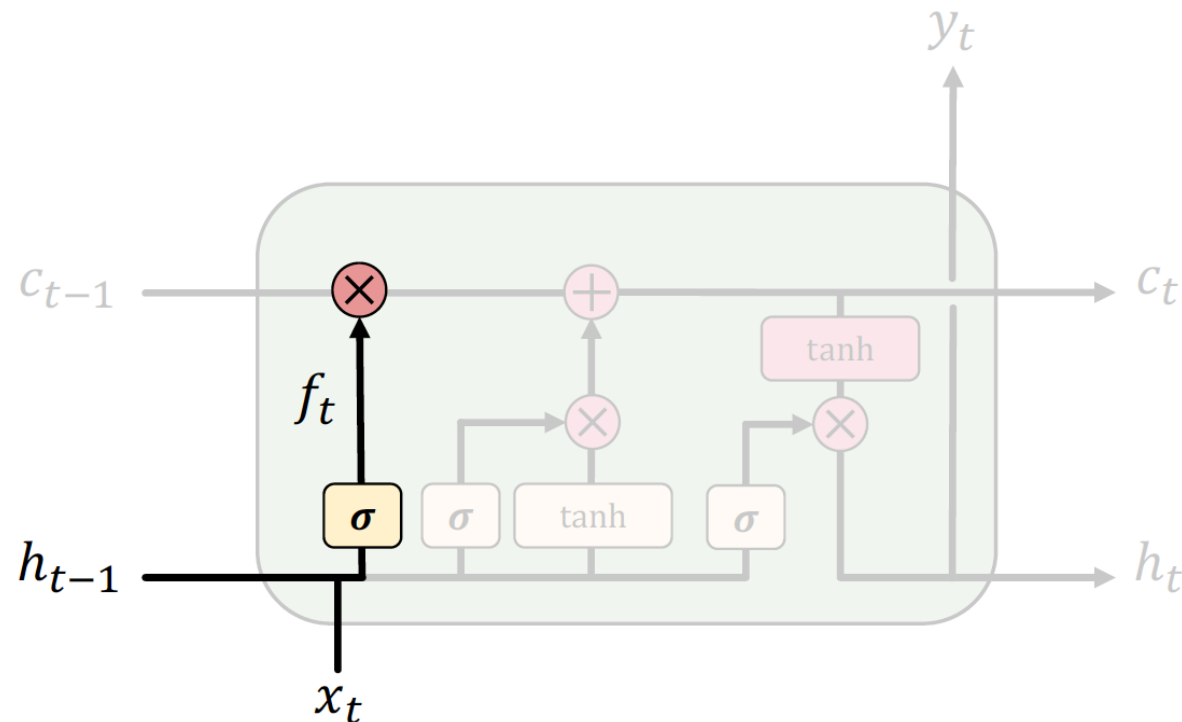
1) Forget 2) Input 3) Update 4) Output



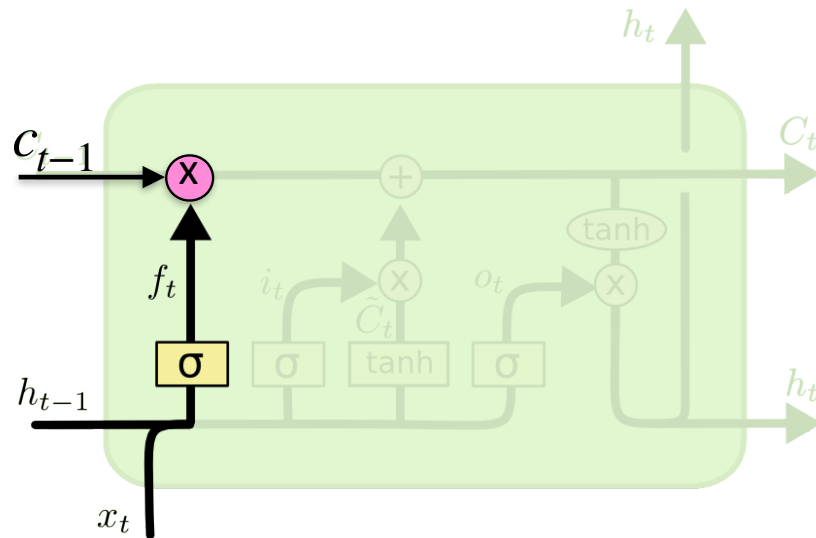
Long Short Term Memory (LSTMs)

1) **Forget** 2) Input 3) Update 4) Output

LSTMs **forget irrelevant** parts of the previous state



Forget gate layer



Forget gate:

- it controls which information to remember and which to forget
- it can also reset the cell state (e.g. the gender of the old subject)

Mathematically:

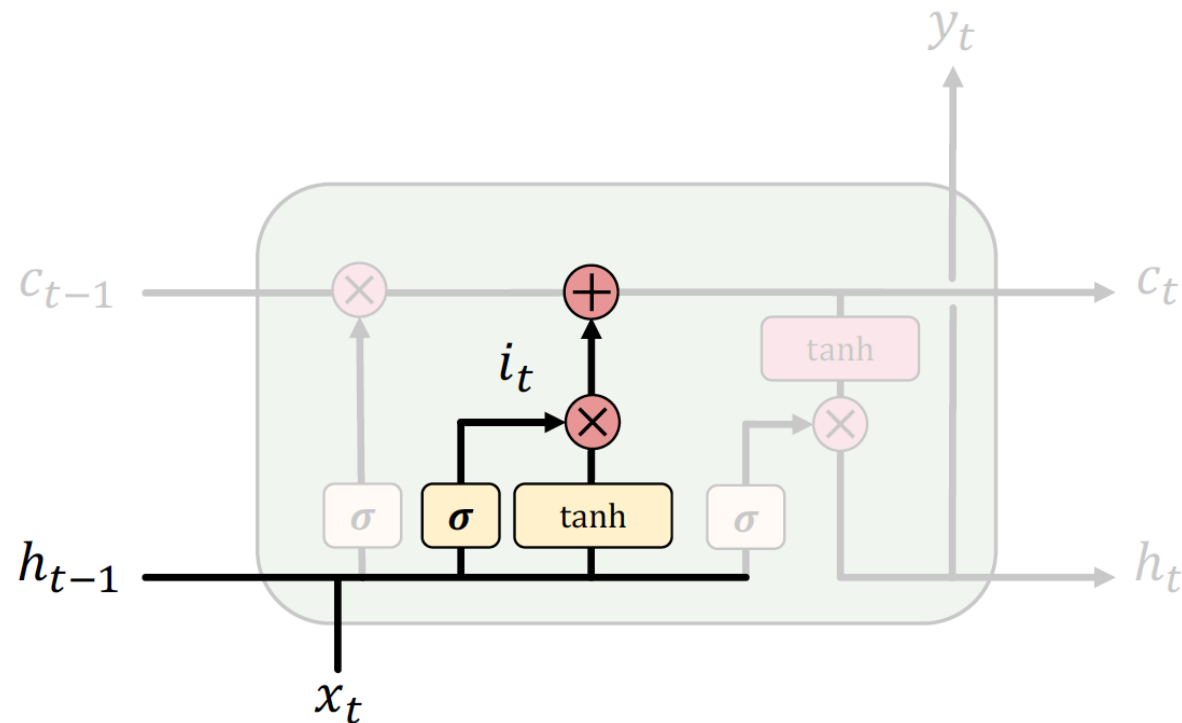
$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

- a Sigmoid σ
- Input: h_{t-1} and x_t
- Output: nb. between 0 and 1:
 - 0: forget
 - 1: remember

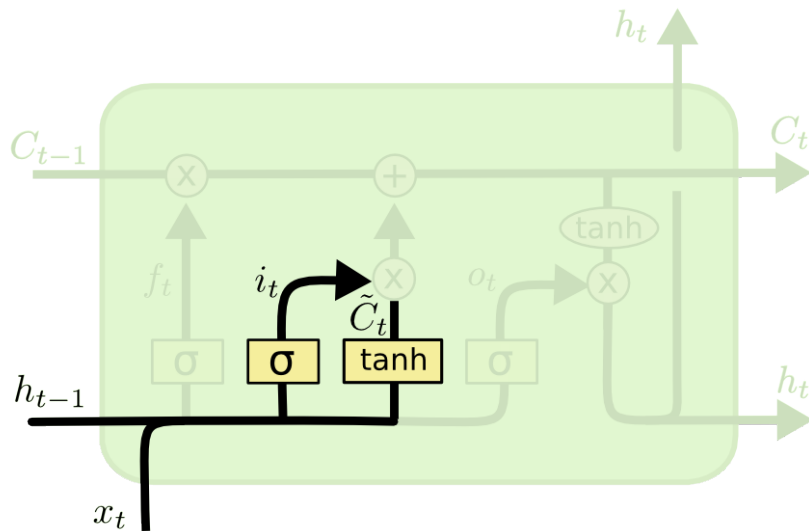
Long Short Term Memory (LSTMs)

1) Forget **2) Input** 3) Update 4) Output

LSTMs **store relevant** new information into the cell state



Input gate layer



Input gate:

- decide what new information to store in the cell state (e.g the gender of the new subject)
- 2 parts

Mathematically:

$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$

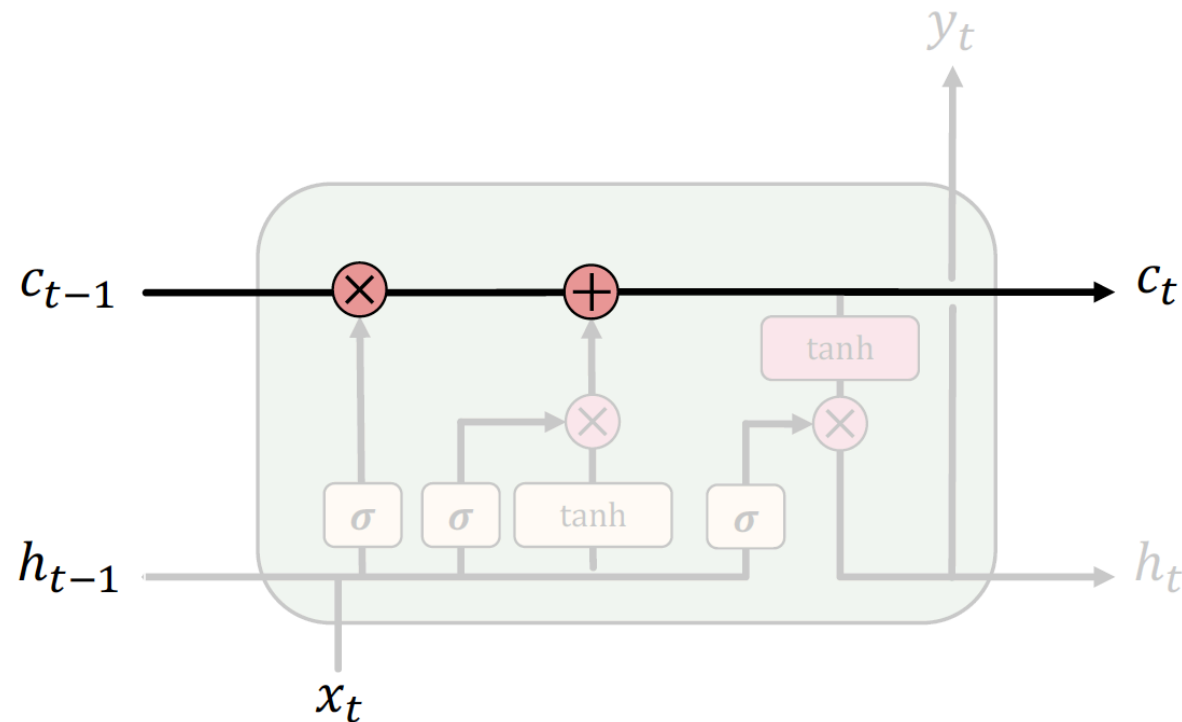
$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- A tanh
(create a new candidate to be possibly added to the state)
- a Sigmoid σ
(to decide which values to update)

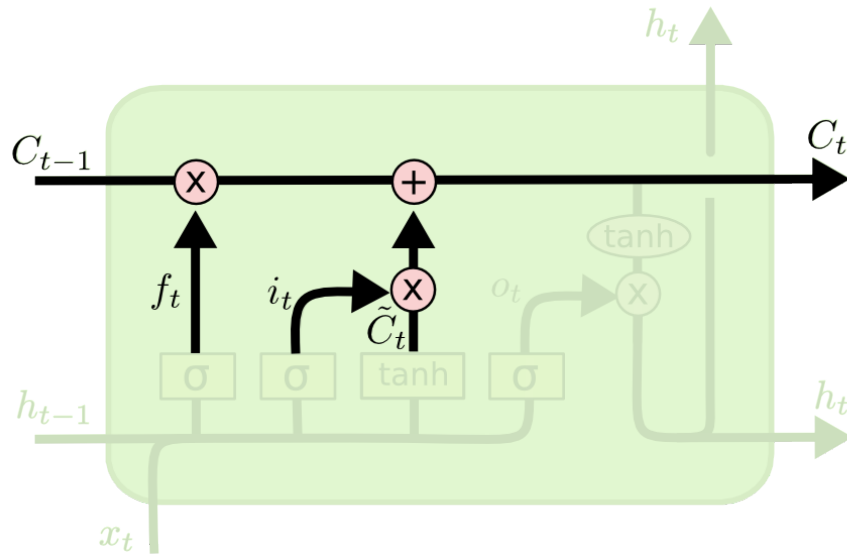
Long Short Term Memory (LSTMs)

1) Forget 2) Input **3) Update** 4) Output

LSTMs **selectively update** cell state values



Cell State update (store)



Cell state update:

- Update C_{t-1} to C_t
- Forget selected old information, write selected new information

Mathematically:

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

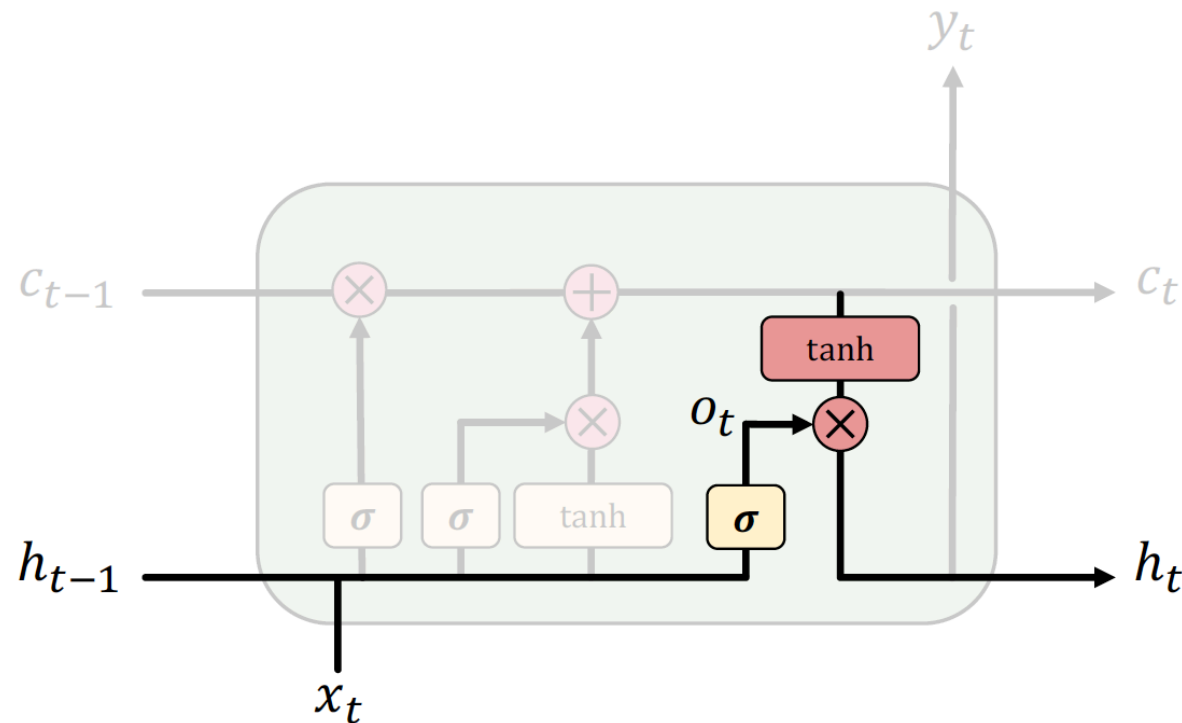
Forget old
irrelevant
information

Add the
weighted new
candidate

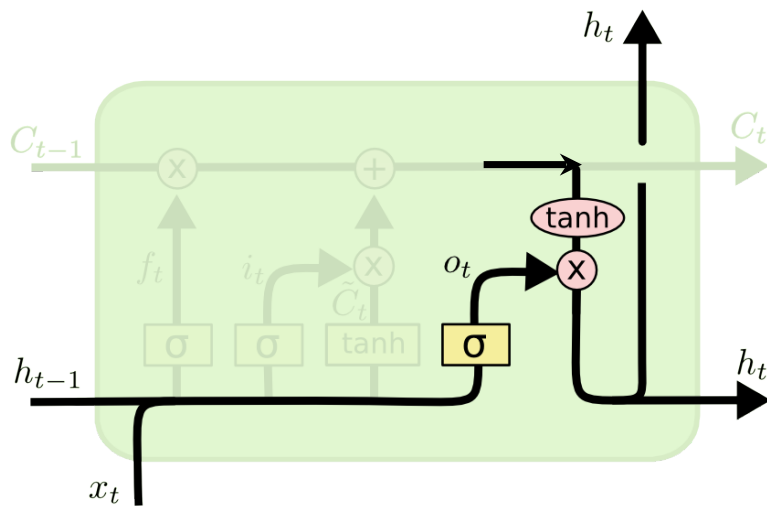
Long Short Term Memory (LSTMs)

1) Forget 2) Input 3) Update 4) **Output**

The **output gate** controls what information is sent to the next time step



Output gate layer



Output gate:

- Output: filtered version of the cell state
- 2 parts

Mathematically:

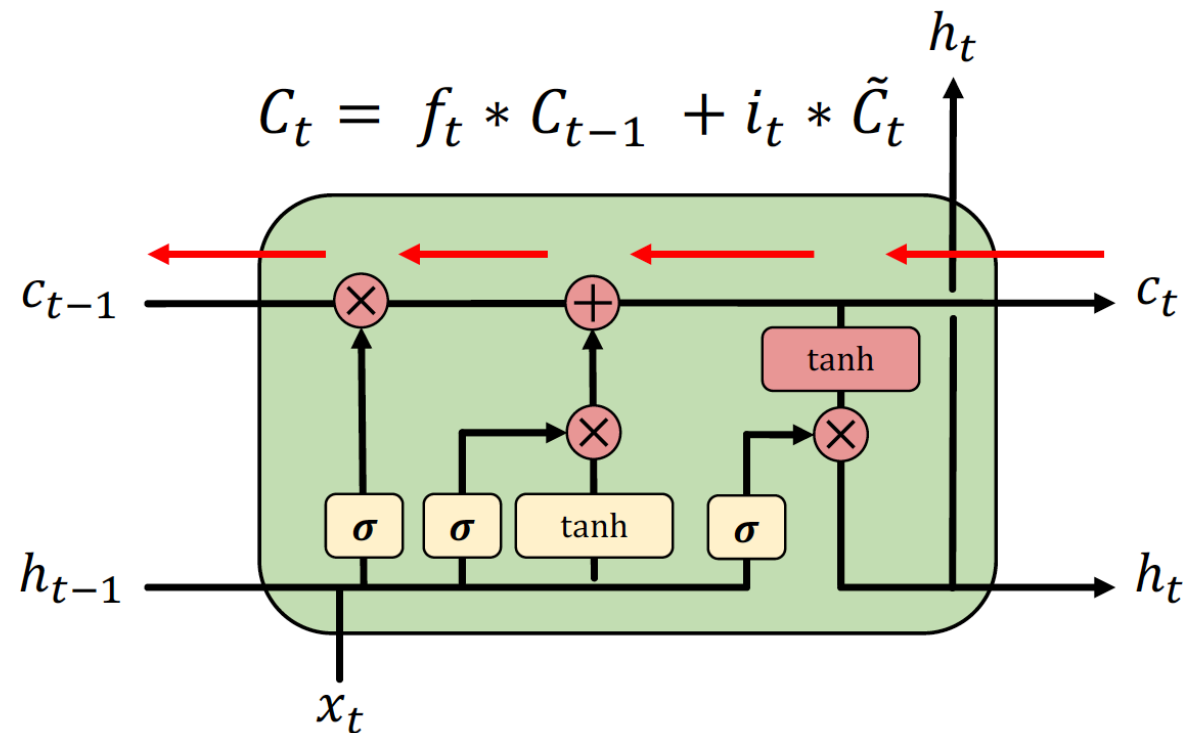
$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$

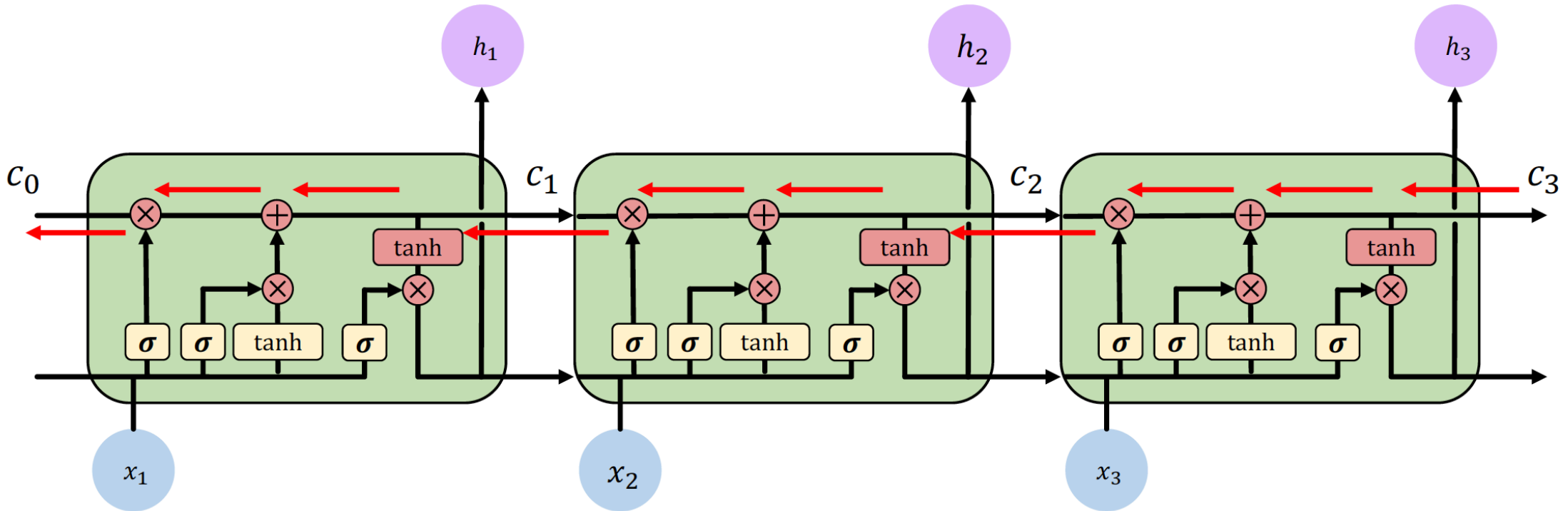
- a Sigmoid σ
(to decide which part of the cell state to output)
- A $\tanh$
(cell state pushed between -1 1)

LSTM gradient flow

Backpropagation from C_t to C_{t-1} requires only elementwise multiplication!
No matrix multiplication $\rightarrow$ avoid vanishing gradient problem.



Uninterrupted gradient flow!



Cell State derivative:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t$$



If $f_t \approx 1$, i.e. don't forget, the derivative is also ≈ 1



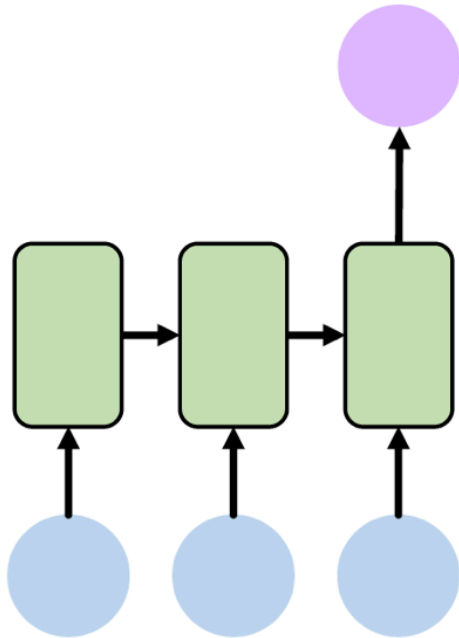
Backpropagation through time:

$$\frac{\partial L}{\partial c_{t-k}} = \frac{\partial L}{\partial c_t} \prod_{m=t-k+1}^t f_m$$

LSTMs: key concepts

1. Maintain a **separate cell state** from what is outputted
2. Use **gates** to control the **flow of information**
 - Forget gate gets rid of irrelevant information
 - Selectively update cell state
 - Output gate returns a filtered version of the cell state

Limitations of Recurrent Models



- Encoding step by step create a bottleneck
- Slow, no parallelization
- Not long memory (cannot process hundreds of thousands sequential information)

Eliminate the need of recurrence

Next
Lecture

- Continuous stream
- Parallelization
- Long memory